Technical Analysis of `memvar.h`: A History-Tracking Value Wrapper for C++20

Rel. 1.4.0

October 14, 2026

Abstract

This report analyses `memvar.h`, a single-header C++20 library that defines two variable wrappers: `mv::memvar<T>`, which behaves like an ordinary variable of type `T` but remembers a bounded number of its previous values, and `mv::memvarTimed<T, Time, Clock>`, which additionally stamps every value with the time at which it was written. The two classes are independent, non-polymorphic types that share one implementation: a history component, `detail::History<Entry>`, which owns the bounded container and every invariant of a write, and a CRTP mixin, `detail::ValueInterface<Derived, T, Entry>`, which provides the whole read, assignment, increment and printing interface. Each history entry of a timed variable holds its value and its time tag together in one container. The report is based exclusively on the code of the header and of the project's unit tests; every code example and every measurement quoted here was compiled and executed against the header, with GCC 13.3 and libstdc++ on x86-64 Linux. The project defines 49 unit tests; all 48 that do not need a one-billion-entry history pass in the two checked safety modes (47 of the 48 defined ones in the default mode, where one SafeRef test does not exist), also under AddressSanitizer and UndefinedBehaviorSanitizer.

The report describes the motivation and the type-constraint system, the history component, the shared interface and the two classes, the free operators and the stream-output helpers, the build-time switch `MEMVAR_SAFEREF_CHECK` that selects how dangling references into the history are handled, and gives complete, runnable usage examples. A dedicated section describes the unit-test suite, in particular the tests that inject an allocation failure and prove that a failed write changes nothing. The report then documents edge cases and closes with a catalogue of limitations, constraints and open issues, for example that the default build gives no protection against references that outlive their history slot. Prerequisites are working knowledge of C++20 templates, concepts and the standard containers.

Contents

1 Introduction and Motivation **4**
- 1.1 The problem . . . 4
- 1.2 Design features . . . 4
- 1.3 Anatomy of the header . . . 4
- 1.4 Methodology and scope . . . 6

2 Type-Constraint System **6**
- 2.1 The building blocks . . . 6
- 2.2 Which types are accepted . . . 7
- 2.3 The auxiliary concepts used by the operators . . . 7

3 The History Component **8**
- 3.1 Capacity constants and printing helper . . . 8
- 3.2 Members and invariants . . . 8

3.2.1 The strong exception guarantee . . . 10
3.2.2 Index convention . . . 11
3.3 Safe access to the current value: `cref()` and `SafeRef` . . . 11
3.3.1 The hazard . . . 11
3.3.2 The three modes . . . 11
3.3.3 How the check works . . . 13
3.3.4 What the handle does and does not protect . . . 13

4 The Shared Interface and the `memvar<T>` Class **14**
4.1 The mixin `detail::ValueInterface` . . . 14
4.1.1 Read path . . . 16
4.1.2 Write path . . . 17
4.1.3 Increment and decrement . . . 17
4.1.4 Printing . . . 17
4.2 The plain variable `memvar<T>` . . . 17

5 The `memvarTimed<T, Time, Clock>` Class **18**
5.1 Purpose and template parameters . . . 18
5.2 State and invariants . . . 20
5.3 The write path . . . 20
5.4 Reading time tags . . . 21
5.5 Special members and assignment . . . 21
5.6 Relation to `memvar<T>` . . . 21
5.7 Choice of Time and Clock . . . 21

6 Free Functions and Namespace-Scope Operator Overloads **22**
6.1 Why a friend struct and a concept . . . 22
6.2 The generated operators . . . 23
6.3 Stream output . . . 24
6.4 Unicode output helpers . . . 25
6.5 Other free functions . . . 26

7 Compilation and Compilation Flags **26**
7.1 Language and library requirements . . . 26
7.2 Build configuration of the project . . . 26
7.3 Rules that follow from the code . . . 27
7.4 Run-time cost . . . 28

8 Complete Usage Examples **28**
8.1 Basic use: history, indexing, printing, clearing . . . 28
8.2 Operators: arithmetic, comparison, compound assignment . . . 29
8.3 Time-tagged variables . . . 31
8.4 Big integers: a Fibonacci recurrence . . . 32
8.5 A realistic pattern: a sensor filter . . . 33
8.6 Dangling-reference protection in the three modes . . . 34
8.7 Strings and Unicode output . . . 35

9 Edge Cases **36**
9.1 Capacity and index errors . . . 39
9.2 Assignment edge cases . . . 39
9.3 Numeric edge cases . . . 39
9.4 Reference-handle edge cases . . . 40
9.5 Failed writes: the strong guarantee . . . 40

9.6 Consequences of the type structure 42
9.7 Stream operator lookup 44
9.8 Unusable but accepted types 44
9.9 Containers and storage of memvars 44

10 The Unit-Test Suite **45**
10.1 Structure and results 45
10.2 Injecting an allocation failure 46
10.3 The failure tests 47
10.4 Gaps and fragilities of the suite 48

11 Limitations, Constraints, and Open Issues **48**
11.1 Constraints of the design 49
11.2 Summary of open issues 49
11.3 Details 50

12 Conclusion **53**

1 Introduction and Motivation

1.1 The problem

Many programs need, besides the current value of a variable, a short memory of what the variable held before: a control loop that needs the previous sample to compute a difference, a recurrence relation such as the Fibonacci numbers whose next term depends on the two previous ones, a monitoring routine that must detect that a reading has jumped, or a debugging session in which the last few assignments to a state variable are of interest. The usual solutions are a hand-written ring buffer next to the variable, or a pair of "current" and "previous" variables that are shifted by hand at every update. Both are error-prone because the bookkeeping is separate from the variable and must be repeated at every assignment site.

`memvar.h` moves that bookkeeping inside the variable. A `memvar<T>` is used almost like a `T`: it is assigned, compared, added, multiplied, incremented and streamed with the same syntax, but every assignment pushes the new value on a bounded history, and the older values stay available by age (`m(1)` is the value before the current one, `m(2)` the one before that, and so on).

1.2 Design features

The following features are implemented.

- **Value-like syntax.** Free operator templates, an implicit conversion `operator T()` and member assignment operators let a `memvar` appear in arithmetic and comparison expressions without explicit unwrapping.
- **Bounded memory.** The history length is fixed at construction and the oldest value is discarded when the capacity is exceeded.
- **Restricted value types.** Only integral (except `bool`), floating-point, the five standard string types, and a designated big-integer type are accepted; the restriction is expressed as a C++20 concept (Section 2).
- **Optional time tagging with one implementation.** `memvar` and `memvarTimed` are separate classes, but neither re-implements anything: the history, the write path, the read path and the printing live in `detail::History` and `detail::ValueInterface`, and a class only supplies what an entry is and what else a write must store (Sections 3 and 4).
- **Failed writes change nothing.** A write is a single insertion into a single container, so an exception (for instance `std::bad_alloc`) leaves the variable, its history and its time tags exactly as they were (Section 3).
- **Cheap, optional protection against dangling references.** The accessor `cref()` can return a checked handle (`SafeRef`) whose cost is one load and one comparison, or a plain `const T&` with no overhead at all; the choice is made at build time (Section 3.3).
- **No run-time polymorphism.** Nothing in the header is `virtual`; the shared interface is a CRTP mixin, so a variable carries no virtual-table pointer.
- **Single header, no external dependency** apart from the standard library (and, for the big-integer case, a forward declaration of `bip::bigint`).
- **No thread safety.** Nothing in the header is synchronised.

1.3 Anatomy of the header

The header (644 lines) has the following parts, listed in the order in which they appear.

Part	Lines	Content
Configuration macros	20–49	`MEMVAR_SAFEREF_ABORT/THROWS/NO_CHECK`, default selection, `#error` on an invalid value
`mv::detail` (text)	54–106	UTF-8 encoder, `toUtf8<CharT>`, `throwIndexOutOfRange`
Global stream operators	108–117	`operator«` for `wstring`, `u16string`, `u32string`, `u8string`
Traits and concepts	119–148	`is_bigint`, `IsAnyOf`, `AnyStandardString`, `MemVarType`
Error support	150–171	`expired_history_reference`, abort path (only if checking is enabled)
`mv::detail` (history)	174–317	capacity constants and validation, `printValues`, `SafeRef`, `History<Entry>`
Forward declarations	319–322	`memvar<T>`, `memvarTimed<T, Time, Clock>`
Operator plumbing	324–355	`memvarAccess`, `IsMemvar`, `valueOf`, `MemvarOperands`
Entry type and mixin	357–467	`detail::TimedEntry`, `detail::ValueInterface`
`memvar<T>`	469–499	the plain variable
`memvarTimed`	501–581	the time-tagged variable
Free functions	583–596	`getHistoryValue`, `getHistoryTimedValue`
Operator templates	598–634	macro-generated binary, comparison and compound-assignment operators
Global stream operators	636–644	`operator«` for `memvar` and `memvarTimed`

Table 1: Layout of `memvar.h`.

The class relationship is as follows:

Listing 1: Class relationship (simplified).

```
mv::detail::History<Entry>                    <- bounded std::deque<Entry>,
    capacity, write counter
mv::detail::SafeRef<T>                        <- checked handle; exists only if
    MEMVAR_SAFEREF_CHECK != NO_CHECK

mv::detail::ValueInterface<Derived, T, Entry>   <- CRTP mixin: owns a History<
    Entry>; read, assign, ++/--, print
        ^ public                                        ^ public
        |                                               |
mv::memvar<T>                                 mv::memvarTimed<T, Time, Clock>
   Entry = T                                     Entry = detail::TimedEntry<T,
       time_point<Clock, Time>>
   private hook  setValue(U&&)                   private hook  setValue(U&&)   (
       adds the time tag)
   clearHistory()                                clearHistory(), memvarEpoch_,
       getTimeTag, timed printing

mv::memvarAccess                              <- friend of both classes; lets
    the free operators read/write
```

An important consequence, examined in Section 5, is that `memvarTimed<int>` *is not* a `memvar<int>`: the two classes instantiate the same mixin with different arguments and are unrelated types. Generic code takes either a template parameter constrained by `mv::IsMemvar` or uses the free operators.

1.4 Methodology and scope

The analysis is static (reading the code) plus dynamic (compiling and running probes). All numbers come from one machine (a virtual machine, GCC 13.3, libstdc++, `-std=c++20 -O3 -march=native -DNDEBUG` unless stated); sizes and allocation counts depend on the standard library implementation and are labelled as such. Statements that were derived by inspection only and not reproduced are marked *(by inspection)*. The header, `bigint.h`, `bigint.cpp`, the project's `CMakeLists.txt` and the unit-test file `unitTests.cpp` are the versions this report describes. The project's own unit tests (49 tests; 48 in the checked modes and 47 in the default mode are run, because `memVarTest.test_6` allocates a history of one billion entries) were run in all three `MEMVAR_SAFEREF_CHECK` modes and pass; the same sources built with `-fsanitize=address,undefined` also pass, and so do all example programs of this report.

2 Type-Constraint System

2.1 The building blocks

Listing 2: Traits and concepts of the constraint system (memvar.h).

```
121 template <typename T>
122 struct is_bigint : std::false_type {};
123
124 template <>
125 struct is_bigint<bip::bigint> : std::true_type {};
126
127 template <typename T>
128 inline constexpr bool is_bigint_v = is_bigint<T>::value;
129
130 template <typename T, typename... Types>
131 concept IsAnyOf = (std::same_as<T, Types> || ...);
132
133 template <typename T>
134 concept AnyStandardString = IsAnyOf<std::remove_cvref_t<T>,
135                                     std::string,
136                                     std::wstring,
137                                     std::u8string,
138                                     std::u16string,
139                                     std::u32string>;
140
141 // Types allowed as memvar<T>'s T: integral (not bool), floating point,
142 // standard string, or bigint.
143 template <typename T>
144 concept MemVarType = (std::is_integral_v<std::remove_cvref_t<T>> &&
145                       !std::same_as<std::remove_cvref_t<T>, bool>) ||
146                       std::is_floating_point_v<std::remove_cvref_t<T>> ||
147                       AnyStandardString<T> ||
148                       is_bigint_v<std::remove_cvref_t<T>>;
```

`is_bigint<T>` / `is_bigint_v<T>`
: A trait that is false for every type except the explicit specialisation for `bip::bigint`. The specialisation needs only the *incomplete* type, which is why the header contains the forward declaration `namespace bip { class bigint; }` (line 52) and does not include `bigint.h`. Note that `bigint.h` defines its own, identical trait `bip::is_bigint`; the two are unrelated.

`IsAnyOf<T, Types...>`
: A fold expression over `std::same_as`: true if `T` is exactly one of the listed types.

`AnyStandardString<T>`
: True for `std::string`, `std::wstring`, `std::u8string`, `std::u16string` and `std::u32string`

(cv-qualifiers and references are removed first). It is used twice: as part of `MemVarType`, and as the discriminator that disables ++ and - for strings.

`MemVarType<T>`
The constraint on the first template parameter of `memvar`, `memvarTimed` and `detail::ValueInterface`: an integral type other than `bool`, *or* a floating-point type, *or* a standard string, *or* a big-integer type.

2.2 Which types are accepted

Type `T`	`MemVarType<T>`	**Observed result of** `memvar<T>`
`int`, `long`, `unsigned`, `size_t`, `int8_t`	satisfied	works
`char`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`	satisfied (they are integral)	`memvar<char>` compiles; values print as characters
`float`, `double`, `long double`	satisfied	works
`std::string`, `wstring`, `u8string`, `u16string`, `u32string`	satisfied	works; ++, - unavailable, and so are -, *, / and their compound forms
`bip::bigint`	satisfied via `is_bigint`	works if `bigint.h` is included first
`bool`	rejected	"template constraint failure"
`std::string_view`, `const char*`, enums, `std::complex`, `std::pmr::string`	rejected	"template constraint failure"
`const int`	**satisfied** (cv-qualifiers are stripped)	hard error inside `std::deque` ("must have a non-const, non-volatile value_type")
`int&`	**satisfied** (references are stripped)	hard errors: the two member assignment operators of the mixin collide (`operator=(T&&)` and `operator=(const T&)` become the same signature), and the allocator cannot form a pointer to a reference

Table 2: Effect of the constraint on representative types (acceptance verified with `static_assert`; diagnostics from GCC 13.3).

The last two rows show a mismatch between the concept and the classes: the concept applies `std::remove_cvref_t` to `T`, but the mixin and the history store `T` (inside a `std::deque`) unmodified. A constraint that rejected cv-qualified and reference types would have produced a clear diagnostic at the point of use instead of an error inside the standard library (see issue I8).

2.3 The auxiliary concepts used by the operators

Three more concepts support the free operators of Section 6 and are described there: `IsMemvar<X>`, `MemvarOperands<Op, L, R>` and the helper function `valueOf`.

3 The History Component

Everything that stores, evicts and indexes values lives in one class template, `detail::History<Entry>`, so that every invariant exists exactly once. `Entry` is the type of one history element: the bare value (for `memvar`) or a value with its time tag (for `memvarTimed`).

3.1 Capacity constants and printing helper

Listing 3: Capacity constants, validation and the printing helper (memvar.h).

```
174 namespace detail {
175 // Smallest history capacity a variable accepts, and the default one.
176 inline constexpr int64_t minimumHistoryCapacity {2};
177 inline constexpr int64_t defaultHistoryCapacity {10};
178
179 // Used in the member-initialiser list, so it runs (and can throw) before the
180 // history container is constructed.
181 inline int64_t validateHistoryCapacity(const int64_t historyCapacity) {
182   if ( historyCapacity >= minimumHistoryCapacity ) return historyCapacity;
183   throw std::invalid_argument("ERROR: The history capacity must be " +
184                               std::to_string(minimumHistoryCapacity) + " at
                                    least, or more");
185 }
186
187 // Prints "[ v v v ]\n"; the values are given by the two iterators.
188 template <typename It, typename Proj>
189 void printValues(std::ostream& os, const std::string& separator, It first, It
      last, Proj value) {
190   os << "[ ";
191   for ( ; first != last; ++first ) os << value(*first) << separator;
192   os << "]\n";
193 }
```

- The minimum capacity is 2 and the default is 10 (both `int64_t`).
- `validateHistoryCapacity` returns its argument if it is at least 2 and otherwise throws `std::invalid_argument` with the message `ERROR: The history capacity must be 2 at least, or more`.
- `printValues` writes `"[ "`, then each value followed by the separator, then `"]\n"`; the values are produced by a projection, so one function serves both entry types and both directions (the iterators are forward or reverse).

3.2 Members and invariants

Listing 4: The History component (memvar.h).

```
239 // The history shared by memvar and memvarTimed: a bounded deque of entries,
240 // newest at the front, plus the capacity and (when SafeRef checking is on)
      the
241 // write counter. Every invariant lives here, once:
242 //   - a write is a single insertion, the only step that can throw, so a
243 //     failed push() or reset() changes nothing (strong exception guarantee);
244 //   - the capacity is validated before the deque exists;
245 //   - indexing is bounds-checked in one place.
246 // What an entry is (a bare value, or a value with its time tag) is the
247 // owner's business: push()/reset() forward their arguments to the Entry
248 // constructor.
249 template <typename Entry>
250 class History {
```

```
public:
  using capacityType = int64_t;

  // validates the capacity first, then stores the first entry
  template <typename... Args>
  explicit History(const capacityType historyCapacity, Args&&... args) :
    capacity_(validateHistoryCapacity(historyCapacity)) {
    memo_.emplace_front(std::forward<Args>(args)...);
  }

  // new newest entry; the oldest one is dropped when the capacity is exceeded
  template <typename... Args>
  void push(Args&&... args) {
    memo_.emplace_front(std::forward<Args>(args)...);
    // nothing below can throw
#if MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
    ++writeCount_;
#endif
    if ( size() > capacity_ ) memo_.pop_back();
  }

  // Restarts from the given entry. It is inserted before the old ones are
  // erased, so a failure leaves the history untouched.
  template <typename... Args>
  void reset(Args&&... args) {
    memo_.emplace_front(std::forward<Args>(args)...);
    memo_.erase(std::next(memo_.begin()), memo_.end());
#if MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
    // every outstanding SafeRef is now dangling: force them all to expire
    writeCount_ += capacity_;
#endif
  }

  // The entry written i writes ago, bounds-checked.
  const Entry& at(const capacityType index) const {
    if ( index >= 0 && index < size() ) {
      return memo_[static_cast<std::size_t>(index)];
    }
    throwIndexOutOfRange(index, memo_.size());
  }

  // Same-expression access only: the reference dies with the next write.
  const Entry& front() const { return memo_.front(); }

  capacityType size() const noexcept { return static_cast<capacityType>(memo_.
      size()); }
  capacityType capacity() const noexcept { return capacity_; }
  bool full() const noexcept { return size() >= capacity_; }
  const std::deque<Entry>& entries() const noexcept { return memo_; }

#if MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
  // handle to a value inside the current front entry
  template <typename T>
  SafeRef<T> safeRef(const T* ptr) const {
    return SafeRef<T>(ptr, writeCount_, capacity_, &writeCount_);
  }
#endif

private:
  // declared first: its initialiser runs, and can throw, before memo_ exists
  const capacityType capacity_;
  std::deque<Entry> memo_ {};
#if MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
```

```
313     // writes ever made (construction is #1); only SafeRef::check() reads it
314     capacityType writeCount_ {1};
315 #endif
316 };
```

The state of a `History` consists of:

- `const capacityType capacity_` – the maximum number of stored entries. `capacityType` is `int64_t`; the code relies on it being signed (the bounds check tests `index >= 0`). The member is `const`, so a variable can never be resized, and it is declared *before* `memo_` so that its initialiser (`validateHistoryCapacity`) runs, and can throw, before the deque is constructed.
- `std::deque<Entry> memo_` – the history itself. The *newest* entry is at the front (index 0) and the oldest at the back. The history always holds the last $\min(\text{writes}, \text{capacity})$ entries.
- `capacityType writeCount_` – present *only* when `MEMVAR_SAFEREF_CHECK` is not `NO_CHECK`. It counts the values ever written (the constructor's first entry is write number 1) and exists solely for `SafeRef` (Section 3.3).

The constructor validates the capacity (in the member initialiser) and then emplaces the first entry from the forwarded arguments; if either step throws, no object exists. The member functions are:

`push(args...)`
: One `emplace_front` of a new newest entry, then (when checking is on) `++writeCount_`, then `pop_back` if the size exceeds the capacity. Writes are amortised $O(1)$.

`reset(args...)`
: Inserts the given entry at the front *first*, then erases everything behind it, then (when checking is on) adds the capacity to `writeCount_`, which expires every outstanding handle.

`at(i)`
: The entry written i writes ago, bounds-checked (`i >= 0 && i < size`); otherwise it calls the `[[noreturn]]` helper `throwIndexOutOfRange`, which throws `std::out_of_range` with the message `Index 5 is out of bounds (size is 2).`

`front()`
: The newest entry, unchecked; the reference dies with the next write.

`size()`, `capacity()`, `full()`, `entries()`
: Observers; `full()` is `size >= capacity`; `entries()` returns the deque by const reference.

`safeRef(ptr)`
: (checked modes only) a handle to a value inside the front entry, see Section 3.3.

3.2.1 The strong exception guarantee

The decisive property of `push` is the order of its steps: the only operation that can throw (allocation of a deque node, or the copy constructor of the value) is the single `emplace_front`. The counter increment and the removal of the oldest element (whose destructor is assumed not to throw) cannot fail. Hence a write either happens completely or not at all. `reset` has the same shape: its insertion precedes the non-throwing erasure. Three consequences follow, all covered by unit tests (Section 10):

- a failed write on a full history does *not* evict the oldest value;
- a failed write does not count in `writeCount_`, so it does not age any outstanding `SafeRef`;
- for a timed variable a value and its tag can never get out of step, because they are one object (Section 5).

3.2.2 Index convention

Index 0 is always the current value; the highest valid index is `getHistorySize() - 1`. The following picture shows a variable with capacity 4 after the writes `0, 10, 20, 30, 40, 50, 60`:

Listing 5: History of a capacity-4 variable after seven writes (0 is the initial value).

```
index         0      1      2      3
memo_      [ 60   |  50   |  40   |  30 ]      <- 10, 20 and the initial 0
    were evicted
           newest                 oldest
front <-- emplace_front(new)                  pop_back() --> back
```

Measured footprint with libstdc++ on x86-64 is given in Table 8. The history is conceptually a ring of `capacity` slots, but the implementation uses a deque and therefore inherits its allocation behaviour (issue I5).

3.3 Safe access to the current value: `cref()` and `SafeRef`

3.3.1 The hazard

A reference to an element of the history is invalidated when that element is evicted. A caller who keeps a `const T&` to the current value across enough writes reads a destroyed object. The internal accessor `getValue()` is protected for that reason, and the public accessor is `cref()`.

3.3.2 The three modes

`MEMVAR_SAFEREF_CHECK` must be defined to one of three values before the header is included (or on the compiler command line); any other value stops compilation with `#error`. When it is undefined the header selects `MEMVAR_SAFEREF_NO_CHECK`.

Macro value	Value	`cref()` returns	On an expired handle
`MEMVAR_SAFEREF_ABORT`	0	`SafeRef`	message to `stderr`, `std::abort()` (noexcept, out of line)
`MEMVAR_SAFEREF_THROWS`	1	`SafeRef`	throws `mv::expired_history_reference` (derives from `std::logic_error`)
`MEMVAR_SAFEREF_NO_CHECK`	2 (default)	`const T&`	nothing: undefined behaviour if the slot was evicted

Table 3: The three safety modes.

Rules of thumb:

- `NO_CHECK` (the default): undefined behaviour arises only if the program uses a reference to an evicted item; use this mode if the code is known never to do so.
- `THROWS`: an expired handle becomes a catchable exception; this is the mode to consider in production code that wants the protection.
- `ABORT`: terminating the process is appropriate for a development or test configuration rather than for production.

Listing 6: Error support for the checked modes (memvar.h).

```
150 #if MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
151 // Thrown by an expired SafeRef when MEMVAR_SAFEREF_CHECK == THROWS.
152 class expired_history_reference : public std::logic_error {
153 public:
```

```
  expired_history_reference() :
    std::logic_error("mv::memvar::cref(): the referenced value has been "
                     "evicted from history (too many writes, or a "
                     "clearHistory(), happened since cref() was called)") {}
};

#if MEMVAR_SAFEREF_CHECK == MEMVAR_SAFEREF_ABORT
// Failure path for MEMVAR_SAFEREF_ABORT. Never throws, so nothing calling it
// needs to be compiled as unwind-safe.
[[noreturn]] inline void reportExpiredHistoryReferenceAndAbort() noexcept {
  std::fprintf(stderr,
               "FATAL: mv::memvar::cref(): the referenced value has been "
               "evicted from history (too many writes, or a clearHistory(), "
               "happened since cref() was called). Terminating.\n");
  std::abort();
}
#endif
#endif  // MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
```

Listing 7: SafeRef (memvar.h).

```
#if MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
// Non-owning handle to the current value at the time cref() was called.
// It remembers the write count at which its slot will be evicted and checks
// that against the live write count of the variable on each dereference, so a
// stale handle fails instead of reading reused deque storage. No allocation,
// no refcounting; a passing check costs one load and one compare. It must not
// outlive the variable itself. Shared by memvar and memvarTimed, which hand
    it
// the address of their own write counter.
template <typename T>
class SafeRef {
public:
  SafeRef(const T* ptr, const int64_t writeSeq, const int64_t historyCapacity,
          const int64_t* writeCount) noexcept :
    ptr_(ptr), expiryWriteCount_(writeSeq + historyCapacity), writeCount_(
        writeCount) {}

  const T& operator*() const noexcept(MEMVAR_SAFEREF_CHECK !=
      MEMVAR_SAFEREF_THROWS) {
    check();
    return *ptr_;
  }
  const T* operator->() const noexcept(MEMVAR_SAFEREF_CHECK !=
      MEMVAR_SAFEREF_THROWS) {
    check();
    return ptr_;
  }
  operator const T&() const noexcept(MEMVAR_SAFEREF_CHECK !=
      MEMVAR_SAFEREF_THROWS) {
    return operator*();
  }

private:
  void check() const noexcept(MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_THROWS) {
    if ( *writeCount_ >= expiryWriteCount_ ) [[unlikely]] {
#if MEMVAR_SAFEREF_CHECK == MEMVAR_SAFEREF_THROWS
      throw expired_history_reference();
#else
      reportExpiredHistoryReferenceAndAbort();
#endif
    }
  }
```

```
232
233   const T* ptr_;
234   int64_t expiryWriteCount_;
235   const int64_t* writeCount_;
236 };
237 #endif  // MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
```

3.3.3 How the check works

A `SafeRef<T>` stores three things: a pointer to the value, a pointer to the owner's `writeCount_`, and `expiryWriteCount_`, computed at issue time as `writeCount_`+`capacity`. Because an entry written as write number w is evicted exactly when write number w+`capacity` happens, the handle is valid as long as the live counter is below that precomputed value (a handle survives `capacity`−1 further writes and expires on the next). Table 4 shows a capacity-3 variable.

Event	`writeCount_`	**History (newest first)**	**Handle on value 1**
construct with 1; `h = m.cref()`	1	1	valid
`m = 2`	2	2 1	valid
`m = 3`	3	3 2 1	valid
`m = 4`	4	4 3 2	**expired** (value 1 evicted)

Table 4: Expiry of a `SafeRef` issued at write number 1 with capacity 3.

`reset` (that is, `clearHistory()`) adds the capacity to `writeCount_`, which pushes the counter past the expiry of every handle issued earlier. A write that throws does not touch `writeCount_` (the increment follows the insertion), so it cannot expire a handle either; this holds in both checked modes and is tested.

Two design points explain the cost. First, `check()` is one load of the counter and one comparison, with `[[unlikely]]` on the failure branch; no allocation, no reference count. Second, the `ABORT` variant reports through a `[[noreturn]] noexcept` non-template function and all `SafeRef` accessors are declared `noexcept(MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_THROWS)`: a call site that cannot throw does not need unwinding tables or register spills, which is what keeps the checked read as cheap as a plain reference (Table 7). The `THROWS` variant gives up that property. The handle is shared by both classes: each hands it the address of its own history's counter, and a handle is 24 bytes.

3.3.4 What the handle does and does not protect

- It protects against *slot eviction* (too many writes, or `clearHistory()`), and a handle *pins the slot*, not the "current value": after a new write the handle still designates the value it was issued for.
- It does *not* protect against destruction of the variable itself (it holds a raw pointer to the counter inside it).
- The conversion `operator const T&()` lets a handle be used where a `const T&` is expected, but implicit conversions do not take part in template argument deduction or member-operator lookup; explicit dereference (`*h`) is the reliable form in such contexts (for instance `*h + "x"` with `std::string`).
- In the default mode there is no handle: `cref()` is `const T&`, and `auto c = m.cref();` silently makes a *copy* of the value, whereas in the checked modes the same line stores a `SafeRef` that can expire (issue I2).
- For a capacity close to the maximum of `int64_t` the expiry arithmetic overflows (Section 9).

4 The Shared Interface and the `memvar<T>` Class

4.1 The mixin `detail::ValueInterface`

Listing 8: The shared interface, first part: types, reading (memvar.h).

```
// The interface shared by memvar and memvarTimed: owns the history and gives
// read access, assignment, ++/--, min/max and printing of the values. Entry is
// T itself (memvar) or TimedEntry<T, ...> (memvarTimed); only its value is used
// here. It is a CRTP mixin, not a polymorphic base: nothing is virtual, and
// Derived is never used through a pointer or reference to it. Derived
// supplies one private hook, setValue(U&&), because only it knows what else a
// write must store (the time tag).
template <typename Derived, MemVarType T, typename Entry>
class ValueInterface {
public:
  // capacityType: this type must be signed
  using capacityType = int64_t;

  // historyValue is a 2-tuple: a T value, and an error flag (false: no error)
  using historyValue = std::tuple<T, bool>;

#if MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
  using SafeRef = detail::SafeRef<T>;
  SafeRef cref() const { return history_.safeRef(&getValue()); }
#else
  const T& cref() const { return getValue(); }
#endif

  capacityType getHistoryCapacity() const noexcept { return history_.capacity(); }
  capacityType getHistorySize() const noexcept { return history_.size(); }
  bool isHistoryFull() const noexcept { return history_.full(); }
  const std::deque<Entry>& getMemVarHistory() const noexcept { return history_.entries(); }

  operator T() const { return getValue(); }
  T operator()() const { return getValue(); }
  T operator()(const capacityType index) const { return valueIn(history_.at(index)); }
  T operator[](const capacityType index) const { return valueIn(history_.at(index)); }
  T valueAt(const capacityType index) const { return valueIn(history_.at(index)); }

  historyValue getHistoryValue(const capacityType index) const {
    return std::make_tuple(valueIn(history_.at(index)), false);
  }

  auto getHistoryMinMax() const {
    const auto [min, max] = std::ranges::minmax_element(history_.entries(),
        std::less<>{}, valueIn);
    return std::make_tuple(valueIn(*min), valueIn(*max));
  }
```

Listing 9: The shared interface, second part: writing, ++/–, printing, protected members (memvar.h).

```
  Derived& operator=(const T& rhs) { derived().setValue(rhs); return derived(); }
  Derived& operator=(T&& rhs) { derived().setValue(std::move(rhs)); return derived(); }
```

```
413
414   // ++mv, --mv return the new value; mv++, mv-- the old one.
415   T operator++() requires (!AnyStandardString<T>) {
416     derived().setValue(static_cast<T>(getValue() + 1));
417     return getValue();
418   }
419   T operator--() requires (!AnyStandardString<T>) {
420     derived().setValue(static_cast<T>(getValue() - 1));
421     return getValue();
422   }
423   T operator++(int) requires (!AnyStandardString<T>) {
424     T old {getValue()};
425     derived().setValue(static_cast<T>(old + 1));
426     return old;
427   }
428   T operator--(int) requires (!AnyStandardString<T>) {
429     T old {getValue()};
430     derived().setValue(static_cast<T>(old - 1));
431     return old;
432   }
433
434   // newest value first
435   void printHistoryData(std::ostream& os = std::cout, const std::string&
        separator = " ") const {
436     const auto& h {history_.entries()};
437     printValues(os, separator, h.cbegin(), h.cend(), valueIn);
438   }
439   // oldest value first
440   void printReverseHistoryData(std::ostream& os = std::cout, const std::string
        & separator = " ") const {
441     const auto& h {history_.entries()};
442     printValues(os, separator, h.crbegin(), h.crend(), valueIn);
443   }
444
445 protected:
446   // the history is built from the arguments of its first entry
447   template <typename... Args>
448   explicit ValueInterface(const capacityType historyCapacity, Args&&...
        firstEntry) :
449     history_(historyCapacity, std::forward<Args>(firstEntry)...) {}
450
451   // not deletable through this type; Derived's own destructor is trivial
452   ~ValueInterface() = default;
453
454   static const T& valueIn(const Entry& e) noexcept {
455     if constexpr ( std::is_same_v<Entry, T> ) return e;
456     else return e.value;
457   }
458
459   const T& getValue() const { return valueIn(history_.front()); }
460
461   History<Entry> history_;
462
463 private:
464   Derived& derived() noexcept { return static_cast<Derived&>(*this); }
465 };
466
467 }  // namespace detail
```

`ValueInterface<Derived, T, Entry>` is a CRTP mixin: it is derived from by exactly one class, which passes itself as `Derived`. Nothing is virtual, and the mixin is never used through a pointer or reference to it (its constructor and destructor are protected, which the edge-case program checks:

std::is_destructible_v of the mixin is false). It owns the only data member, History<Entry> history_, and obtains from Derived one private hook, setValue(U&&), because only the derived class knows what else a write must store. The mixin reaches it through the private derived() cast, which is possible because each class declares the mixin instance as a friend.

Member	Meaning
operator=(const T&), operator=(T&&)	record a new value; return Derived&, so assignments chain
operator T(), operator()()	current value, *by copy*
operator()(i), operator[](i), valueAt(i)	the value written i writes ago (0 = newest), by copy; std::out_of_range if $i < 0$ or $i \geq$ size
getHistoryValue(i)	std::tuple<T,bool>: the value and an error flag (always false); for a timed variable the tag is *not* part of it
getHistoryMinMax()	std::tuple<T,T> of the smallest and largest stored *value* (std::ranges::minmax_element with a projection to the value, so the tag of a timed entry plays no part)
getHistorySize(), getHistoryCapacity(), isHistoryFull()	current number of stored values, capacity, size $\geq$ capacity (all noexcept)
getMemVarHistory()	const std::deque<Entry>&: deque<T> for a plain variable and deque<detail::TimedEntry<T, time_point» for a timed one
++, - (prefix/postfix)	requires (!AnyStandardString<T>); prefix returns the *new* value, postfix the *old* one, both by value
printHistoryData(os, sep), printReverseHistoryData(os, sep)	newest-to-oldest / oldest-to-newest dump of the values
cref()	read access to the current value; return type depends on MEMVAR_SAFEREF_CHECK
historyValue, capacityType, SafeRef	type aliases (SafeRef only in the checked modes)

Table 5: Public members common to memvar<T> and memvarTimed<T, . . . > (arithmetic and comparison operators are free functions, Section 6).

4.1.1 Read path

- getValue() (protected) returns the value of the front entry by reference through valueIn, a static helper that returns the entry itself for a plain variable and its .value member for a timed one; it is the primitive used inside the library.
- operator T() and operator()() return T by value.
- operator()(i), operator[](i), valueAt(i) and getHistoryValue(i) all go through History::at, which performs the bounds check. The time-tag accessors of memvarTimed use the same at and therefore report the same message.
- getMemVarHistory() returns a const reference to the whole deque: this is the only way to iterate without copying, and the reference to the removed element is invalidated by the next write. Its element type is Entry, so for a timed variable the caller sees detail::TimedEntry objects.

The free function mv::getHistoryValue(const memvar<T>&, index) returns the *value only* (it is mv(index)), whereas the member getHistoryValue(index) returns a tuple<T,bool>; the same name therefore has two different return types depending on how it is called.

4.1.2 Write path

All writes funnel through the derived class's `setValue(U&&)` (`U` is `const T&` for a copy or `T` for a move, so an rvalue assigned to a `memvar<std::string>` is moved, not copied), which calls `History::push`. The assignment operators of the mixin, the four increment/decrement operators, the copy assignment of each class and the compound assignments of Section 6 all end there. The compound assignments are free functions that first compute the new value and only then call `setValue`; so if the arithmetic throws (for example a `bigint` division by zero) nothing is recorded (Section 8).

4.1.3 Increment and decrement

The four operators are constrained with `requires (!AnyStandardString<T>)` on a non-template member function, a C++20 feature that makes the member *unavailable* rather than ill-formed for string types. The result is computed as `static_cast<T>(value ± 1)`, which keeps narrow types such as `unsigned char` well-defined (they are promoted to `int` for the addition and truncated back). Each function returns `T` *by value*, not the variable; hence, unlike the built-in operators, `++m = 3` is not valid. The postfix forms copy the old value before writing and return it. For a signed integral `T` that is not narrower than `int`, incrementing the maximum value is signed overflow, which is undefined behaviour *(by inspection)*.

4.1.4 Printing

`printHistoryData` and `printReverseHistoryData` call `printValues` with forward or reverse iterators over the entries and the `valueIn` projection. With the default separator the output of a three-element history is `[ 3 2 1 ]`; there is a trailing separator before the closing bracket. Values are streamed with `operator«` found by ordinary unqualified lookup, so wide strings use the UTF-8 helpers of Section 6.

4.2 The plain variable `memvar<T>`

Listing 10: The plain variable (memvar.h).

```
469 // memvar
470 // a variable with memory of old values
471 // only strings, integral or floating point types allowed
472 // The history is a std::deque<T>, newest value at the front.
473 template <MemVarType T>
474 class memvar : public detail::ValueInterface<memvar<T>, T, T> {
475   using Base = detail::ValueInterface<memvar<T>, T, T>;
476   friend Base;
477   friend struct memvarAccess;
478 public:
479   using typename Base::capacityType;
480   using Base::operator=;
481
482   memvar() : memvar(T{}) {}
483
484   explicit memvar(const T& value, const capacityType historyCapacity = detail
          ::defaultHistoryCapacity) :
485     Base(historyCapacity, value) {}
486
487   memvar(const memvar&) = delete;
488   memvar(memvar&&) = delete;
489   memvar& operator=(memvar&&) = delete;
490   memvar& operator=(const memvar& rhs) { setValue(rhs.getValue()); return *
          this; }
491
492   // Restarts from T{}.
493   void clearHistory() { this->history_.reset(T{}); }
494
```

```
private:
  // U is const T& (copy) or T (move)
  template <typename U>
  void setValue(U&& value) { this->history_.push(std::forward<U>(value)); }
};  // class memvar
```

- The class adds no data member: `sizeof(memvar<int>)` equals that of its `History` (88 bytes in the default mode, 96 in the checked modes, Table 8). It is not polymorphic and not `final`.
- The default constructor delegates to the value constructor with `T{}` and the default capacity, so there is a single place that stores the first value.
- The value constructor is `explicit`: `memvar<int> m = 5;` does not compile, `memvar<int> m(5);` and `memvar<int> m{5};` do. It hands the capacity and the first value to the mixin, which builds the history: the capacity is validated first.
- `using Base::operator=` brings the two value assignments of the mixin into scope; the class itself declares the copy assignment, which copies only the *current value* of the right-hand side as a new write (history and capacity are never copied).
- Copy construction, move construction and move assignment are deleted (confirmed with `static_asserts` in the edge-case program of Section 9). A consequence is that `a = std::move(b)` between two variables does not compile, because the deleted `operator=(memvar&&)` is the best match.
- `clearHistory()` calls `History::reset(T{})`: the history shrinks to a single value-initialised element, inserted before the old ones are erased, so a failed clear leaves the history untouched. The current value is therefore *reset to zero/empty string*, not kept (issue I10). When checking is on every outstanding `SafeRef` expires.
- `setValue` is private; the mixin and `memvarAccess` are friends.

5 The `memvarTimed<T, Time, Clock>` Class

5.1 Purpose and template parameters

`memvarTimed` stores, for every value in the history, the time at which the value was written. The two extra template parameters default to `std::chrono::nanoseconds` (`Time`, the resolution of the tags) and `std::chrono::steady_clock` (`Clock`, the time source). It derives from `ValueInterface` with `Entry = detail::TimedEntry<T, time_point>`.

Listing 11: One history entry of a timed variable (memvar.h).

```
namespace detail {

// One history entry of a memvarTimed: a value and the time it was written.
template <typename T, typename TimePoint>
struct TimedEntry {
  template <typename U>
  TimedEntry(U&& v, const TimePoint t) : value(std::forward<U>(v)), tag(t) {}
  T value;
  TimePoint tag;
};
```

Listing 12: The time-tagged variable (memvar.h).

```
// memvarTimed
```

```cpp
// Same as memvar, but every value also carries the time of its write. The
// history is a std::deque<Entry>; an Entry holds a value and its time point
// together, so the two are inserted and evicted as one unit and can never get
// out of step, whether a write succeeds or throws.
template <MemVarType T, typename Time, typename Clock>
class memvarTimed : public detail::ValueInterface<memvarTimed<T, Time, Clock>, T,
                        detail::TimedEntry<T, std::chrono::time_point<Clock, Time>>> {
public:
  using time_point = std::chrono::time_point<Clock, Time>;
  using Entry = detail::TimedEntry<T, time_point>;

private:
  using Base = detail::ValueInterface<memvarTimed, T, Entry>;
  friend Base;
  friend struct memvarAccess;

public:
  using typename Base::capacityType;
  using Base::operator=;

  // a T value, the time tag, and an error flag (false: no error)
  using historyTimedValue = std::tuple<T, Time, bool>;

  memvarTimed() : memvarTimed(T{}) {}

  // the epoch is the time of the first value
  explicit memvarTimed(const T& value, const capacityType historyCapacity =
      detail::defaultHistoryCapacity) :
    Base(historyCapacity, value, now()),
    memvarEpoch_(this->history_.front().tag) {}

  memvarTimed(const memvarTimed&) = delete;
  memvarTimed(memvarTimed&&) = delete;
  memvarTimed& operator=(memvarTimed&&) = delete;
  memvarTimed& operator=(const memvarTimed& rhs) { setValue(rhs.getValue());
      return *this; }

  // time elapsed from the memvar's epoch (the time of its first value) to the
  // write of the i-th value
  Time getTimeTag(const capacityType index = 0) const {
    return std::chrono::duration_cast<Time>(this->history_.at(index).tag -
        memvarEpoch_);
  }

  historyTimedValue getHistoryTimedValue(const capacityType index) const {
    const Entry& e {this->history_.at(index)};
    return std::make_tuple(e.value, std::chrono::duration_cast<Time>(e.tag -
        memvarEpoch_), false);
  }

  void printHistoryTimedData(std::ostream& os = std::cout, const std::string&
      separator = "\n") const {
    printTimed(os, separator, false);
  }
  void printReverseHistoryTimedData(std::ostream& os = std::cout, const std::
      string& separator = "\n") const {
    printTimed(os, separator, true);
  }

  // Restarts from T{}; the new entry's time becomes the new epoch.
  void clearHistory() {
```

```
557     this->history_.reset(T{}, now());
558     memvarEpoch_ = this->history_.front().tag;
559   }
560
561 private:
562   static time_point now() { return std::chrono::time_point_cast<Time>(Clock::
          now()); }
563
564   // U is const T& (copy) or T (move)
565   template <typename U>
566   void setValue(U&& value) { this->history_.push(std::forward<U>(value), now()
          ); }
567
568   void printTimed(std::ostream& os, const std::string& separator, const bool
          reverse) const {
569     const auto& h {this->history_.entries()};
570     const std::size_t n {h.size()};
571     os << "{ --- begin ---\n[TimeTag:Value]\n";
572     for ( std::size_t k {0}; k < n; ++k ) {
573       const Entry& e {h[reverse ? n - 1 - k : k]};
574       os << "[" << std::chrono::duration_cast<Time>(e.tag - memvarEpoch_).
              count()
575          << ":" << e.value << "]" << separator;
576     }
577     os << "  --- end --- }\n\n";
578   }
579
580   time_point memvarEpoch_;
581 };  // class memvarTimed
```

5.2 State and invariants

- The history is the mixin's `std::deque<TimedEntry<T, time_point>>`: each element is one value together with the `time_point` at which it was written (`TimedEntry` is a plain pair with a converting constructor `TimedEntry(U&&, TimePoint)`; the public aliases `Entry` and `time_point` name its type and the tag type). There is no parallel container, so the invariant "one tag per value" holds by construction and needs no maintenance.
- `memvarEpoch_` – the only member added by `memvarTimed`. The constructor sets it to the tag of the first entry, which the base has just stored. Reported tags are offsets from the epoch, so the first value has the tag zero *exactly* (not merely approximately).
- Time tags are stored with the resolution `Time` (`now()` truncates the clock reading with `time_point_cast`); the accessors apply `duration_cast<Time>` to the difference between a tag and the epoch.
- `sizeof(memvarTimed<int>)` is 96 bytes in the default mode and 104 in the checked modes; an entry of `int` is 16 bytes.

5.3 The write path

`setValue` calls `History::push(value, now())`: one clock read and one insertion (about 23–24 ns on the test machine, Table 7, nearly all of it the clock call). Because insertion is the only step that can throw (Section 3), a timed write has the strong exception guarantee, and the value and tag of every entry are inserted and evicted as one unit. The clock is read as an argument, i.e. *before* the entry is constructed (issue I13); the same holds in the constructor.

5.4 Reading time tags

- `getTimeTag(index = 0)` returns the duration from the epoch to the write of the entry, `duration_cast` to `Time`. The parameter is the signed `capacityType` and the entry is fetched with the same bounds-checked `History::at` as the value accessors: a negative or too large index throws `std::out_of_range` with the identical message `Index` *i* `is out of bounds (size is` *n*`)`.
- `getHistoryTimedValue(index)` returns `std::tuple<T, Time, bool>` built from one entry (one lookup, one check).
- `printHistoryTimedData` and `printReverseHistoryTimedData` share the private `printTimed`: a header (`{ -- begin --`, a column title `[TimeTag:Value]`, one `[tag:value]` item per entry followed by the separator, and the footer `-- end -- }` plus a blank line). The tag is printed with `.count()`, i.e. without its unit.
- Streaming a `memvarTimed` with `operator«` prints `[tag-of-newest:current-value]`.

5.5 Special members and assignment

- `clearHistory()` calls `History::reset(T{}, now())` and then makes the new entry's tag the epoch: after a clear the single remaining value has tag exactly zero relative to the *new* epoch, so tags are not comparable across a clear. If `reset` throws, the epoch is not touched and the history is unchanged.
- `using Base::operator=` makes the two value assignments of the mixin visible, and the class declares its own copy assignment, so every assignment returns `memvarTimed&`. No base-class assignment is hidden. Assigning a bare `memvar<T>` to a `memvarTimed` works through the implicit `operator T()` followed by `operator=(T&&)` (one extra copy of `T`); this is exercised by the unit tests.
- Everything else (`operator()`, `operator[]`, `++`, `-`, the history getters, `cref()`, printing of values) is inherited from the mixin. The compound assignments are free templates, so they apply without being redeclared.
- Copy/move construction and move assignment are deleted. The class is neither polymorphic nor `final`.

5.6 Relation to `memvar<T>`

The two classes instantiate the same mixin with different arguments and are *unrelated types*: `memvarTimed<int>` does not convert to `const memvar<int>&`, and `memvarTimed<int, ms>` and `memvarTimed<int, ns>` do not share a base either. Code that must accept both flavours is written as a template over a type satisfying `mv::IsMemvar`, or in terms of the free operators (Listing 42, issue I17).

5.7 Choice of Time and Clock

`now()` is `time_point_cast<Time>(Clock::now())`. Consequently

- a coarser `Time` (milliseconds) truncates; a finer one (picoseconds) is allowed and merely scaled;
- a floating-point duration such as `duration<double>` works and its `.count()` is a `double` (verified by the example in Section 8);
- a non-monotonic clock such as `system_clock` may produce tags that decrease with a clock adjustment, since nothing in the class enforces monotonicity.

6 Free Functions and Namespace-Scope Operator Overloads

6.1 Why a friend struct and a concept

The operators are *not* members, so that the same code serves `memvar` and `memvarTimed`, and so that both operand orders (`m + 1` and `1 + m`) are covered by one template. They need the protected getter and the private setter, which is granted to a single helper:

Listing 13: Operator plumbing (memvar.h).

```
// Gives the free operators below access to the protected value/setter of both
// kinds of variable.
struct memvarAccess {
  template <MemVarType T>
  static const T& get(const memvar<T>& m) { return m.getValue(); }

  template <MemVarType T, typename Time, typename Clock>
  static const T& get(const memvarTimed<T, Time, Clock>& m) { return m.
      getValue(); }

  template <MemVarType T, typename U>
  static void set(memvar<T>& m, U&& v) { m.setValue(std::forward<U>(v)); }

  template <MemVarType T, typename Time, typename Clock, typename U>
  static void set(memvarTimed<T, Time, Clock>& m, U&& v) { m.setValue(std::
      forward<U>(v)); }
};

template <typename X>
concept IsMemvar = requires(const X& x) { memvarAccess::get(x); };

// The current value if x is a memvar (no copy), otherwise x itself.
template <typename X>
constexpr const auto& valueOf(const X& x) {
  if constexpr ( IsMemvar<X> ) return memvarAccess::get(x);
  else return x;
}

// "L Op R is valid on the underlying values and at least one side is a memvar
    ".
template <typename Op, typename L, typename R>
concept MemvarOperands =
  (IsMemvar<L> || IsMemvar<R>) &&
  std::is_invocable_v<Op, decltype(valueOf(std::declval<const L&>())),
                          decltype(valueOf(std::declval<const R&>()))>;
```

- `memvarAccess` is a `friend` of both classes and has one `get` and one `set` overload per class (`memvar<T>` and `memvarTimed<T, Time, Clock>`), so one set of operators serves both flavours although they are unrelated types. The parameters are `const memvar<T>&` and `const memvarTimed<...>&`, so a class derived from one of them is accepted too, by derived-to-base deduction.
- `IsMemvar<X>` is true exactly when `memvarAccess::get` can be called with an `X`; for any type that is not a memvar, deduction fails and the concept is false.
- `valueOf(x)` returns `const auto&`: the stored current value when `x` is a memvar (no copy), otherwise `x` itself.
- `MemvarOperands<Op, L, R>` states that at least one operand is a memvar and that the standard function object `Op` (`std::plus<>`, `std::less<>`, ...) is invocable on the underlying values. The

concept is a conjunction whose first operand is false for two non-memvar types, so the templates are inert for all unrelated expressions.

6.2 The generated operators

Listing 14: Binary, comparison and compound-assignment operators (memvar.h).

```
598 // Binary operators between a memvar (bare or timed) and another memvar or a
599 // plain value, on either side. One constrained template per operator;
    operands
600 // are read by const reference.
601 #define MEMVAR_BINARY_OPERATOR(OP, FUNCTOR)
                                                \
602   template <typename L, typename R>
                                                      \
603     requires MemvarOperands<FUNCTOR, L, R>
                                                    \
604   auto operator OP(const L& lhs, const R& rhs) {
                                                 \
605     return valueOf(lhs) OP valueOf(rhs);
                                                       \
606   }
607
608 MEMVAR_BINARY_OPERATOR(==, std::equal_to<>)
609 MEMVAR_BINARY_OPERATOR(<,  std::less<>)
610 MEMVAR_BINARY_OPERATOR(>,  std::greater<>)
611 MEMVAR_BINARY_OPERATOR(<=, std::less_equal<>)
612 MEMVAR_BINARY_OPERATOR(>=, std::greater_equal<>)
613 MEMVAR_BINARY_OPERATOR(+,  std::plus<>)
614 MEMVAR_BINARY_OPERATOR(-,  std::minus<>)
615 MEMVAR_BINARY_OPERATOR(*,  std::multiplies<>)
616 MEMVAR_BINARY_OPERATOR(/,  std::divides<>)
617 #undef MEMVAR_BINARY_OPERATOR
618
619 // Compound assignment: the left side must be a memvar; it records a new value
    .
620 #define MEMVAR_COMPOUND_OPERATOR(OP, FUNCTOR)
                                                \
621   template <typename L, typename R>
                                                      \
622     requires (IsMemvar<L> && MemvarOperands<FUNCTOR, L, R>)
                                                \
623   L& operator OP##=(L& lhs, const R& rhs) {
                                                    \
624     memvarAccess::set(lhs, valueOf(lhs) OP valueOf(rhs));
                                                \
625     return lhs;
                                                                                \
626   }
627
628 MEMVAR_COMPOUND_OPERATOR(+, std::plus<>)
629 MEMVAR_COMPOUND_OPERATOR(-, std::minus<>)
630 MEMVAR_COMPOUND_OPERATOR(*, std::multiplies<>)
631 MEMVAR_COMPOUND_OPERATOR(/, std::divides<>)
632 #undef MEMVAR_COMPOUND_OPERATOR
```

Two macros stamp out one function template per operator (they are `#undef`ed afterwards):

Observations:

- Operands may be `memvar`, `memvarTimed`, `T`, or any type for which the underlying operation compiles: `memvar<string> == "abc"`, `"<" + s`, `memvar<bigint> + 7`, and comparisons between different

Operators	Signature (conceptually)	Behaviour
`== < > <= >=`	`bool op(const L&, const R&)`	compare the underlying values; at least one side is a memvar or memvarTimed, the other any type comparable with `T`
`+ - * /`	`auto op(const L&, const R&)`	return a *plain value* of the type of the underlying expression, never a memvar
`+= -= *= /=`	`L& op(L&, const R&)`	`L` must be a memvar; computes the new value, then records it with `memvarAccess::set`; returns the left operand
`!=`	(synthesised)	C++20 rewrites `a != b` as `!(a == b)`; reversed forms of == are also synthesised

Table 6: Operator families.

memvar value types such as `memvar<int> == memvar<long>`.

- Because the result is `auto` of the underlying expression, it is *not* re-narrowed to `T`: `memvar<int8_t> + memvar<int8_t>` is an `int`, `memvar<int> + 1.5` is a `double` (`static_asserted` in the edge-case program).
- Compound assignment writes `valueOf(lhs) OP valueOf(rhs)` into the history: if the type of that result differs from `T` it is converted by `emplace_front`'s direct-initialisation, *silently truncating* (`memvar<int> += 1.9` records 2; `memvar<int8_t>(120) += 20` records −116).
- The operators for `-`, `*`, `/` and their compound forms are absent for strings because the standard function objects are not invocable on strings; no explicit `requires` is needed.
- The body is `valueOf(lhs) OP valueOf(rhs)` written directly, so a compiler diagnostic about the underlying operation (for instance `-Wsign-compare` for mixed signedness) is reported at the macro line inside `memvar.h` with the user's expression only as "required from here" (Section 9).
- The compound templates require `IsMemvar<L>`, which is also satisfied by a *const* variable; the constraint then holds but the body cannot compile (Section 9).
- There are no %, %=, unary `-`, bitwise or shift operators. For arithmetic `T` some of them still compile through the implicit `operator T()` and return a plain value (`m % 3`); for `bigint`, whose % and unary minus are *member* operators, they do not (issue I6).
- The templates live in namespace `mv` and are found by argument dependent lookup.

6.3 Stream output

Listing 15: Stream operators for memvar and memvarTimed (memvar.h).

```
636 template <typename T>
637 std::ostream& operator<<(std::ostream& os, const mv::memvar<T>& mv) {
638   return os << mv::valueOf(mv);
639 }
640
641 template <typename T, typename Time, typename Clock>
642 std::ostream& operator<<(std::ostream& os, const mv::memvarTimed<T, Time,
      Clock>& mvt) {
643   return os << "[" << mvt.getTimeTag().count() << ":" << mv::valueOf(mvt) << "
        ]";
644 }
```

The two operator« templates are declared in the *global* namespace, one for each class. A memvar<T> prints as its current value; a memvarTimed prints as [tag:value]. Because they are in the global namespace they are not found by argument-dependent lookup (the associated namespace of memvar<T> is mv), only by ordinary lookup, and ordinary lookup stops at the first enclosing scope that declares any operator«. Section 9 shows the consequence.

6.4 Unicode output helpers

Listing 16: UTF-8 conversion helpers and the string stream operators (memvar.h).

```
// wstring / u16string / u32string -> UTF-8. 16-bit code units (char16_t, and
// wchar_t on Windows) are combined into surrogate pairs; wider ones are code
// points already.
template <typename CharT>
std::string toUtf8(const std::basic_string<CharT>& s) {
  using Unit = std::make_unsigned_t<CharT>;
  std::string out;
  out.reserve(s.size());
  for ( std::size_t i {0}; i < s.size(); ) {
    char32_t cp {static_cast<Unit>(s[i++])};
    if constexpr ( sizeof(CharT) == 2 ) {
      if ( cp >= 0xD800 && cp <= 0xDBFF && i < s.size() ) {
        const char32_t low {static_cast<Unit>(s[i])};
        if ( low >= 0xDC00 && low <= 0xDFFF ) {
          cp = 0x10000 + ((cp - 0xD800) << 10) + (low - 0xDC00);
          ++i;
        }
      }
    }
    appendUtf8(out, cp);
  }
  return out;
}

[[noreturn]] inline void throwIndexOutOfRange(const int64_t index, const std::
    size_t size) {
  throw std::out_of_range("Index " + std::to_string(index) +
                          " is out of bounds (size is " + std::to_string(size)
                              + ").");
}
}  // namespace mv::detail

// Stream output for the non-char standard strings (UTF-8 on the wire).
template <typename CharT>
  requires (std::same_as<CharT, wchar_t> || std::same_as<CharT, char16_t> ||
      std::same_as<CharT, char32_t>)
  std::ostream& operator<<(std::ostream& os, const std::basic_string<CharT>& s
      ) {
    return os << mv::detail::toUtf8(s);
}

inline std::ostream& operator<<(std::ostream& os, const std::u8string& s) {
  return os << std::string_view(reinterpret_cast<const char*>(s.data()), s.
      size());
}
```

The standard library has no ostream insertion for wstring, u16string, u32string or u8string, and C++20 deleted the overloads for the corresponding character pointers. The header supplies them, in the global namespace, so that memvar<std::wstring> and the others can be printed (and so that the values in a history can be dumped):

- `appendUtf8` (lines 56–76) encodes one code point into 1–4 bytes; surrogates and values above `U+10FFFF` become `U+FFFD`.
- `toUtf8<CharT>` reads each code unit through `std::make_unsigned_t<CharT>`, and, for 2-byte code units (`char16_t` and `wchar_t` where it is 16 bits wide), combines a high surrogate followed by a low surrogate into one code point. A lone surrogate (including a high surrogate at the very end) is replaced by `U+FFFD`.
- the `requires` clause restricts the template to `wchar_t`, `char16_t` and `char32_t`; `u8string` is forwarded byte for byte through a `string_view`.

6.5 Other free functions

Listing 17: Free history accessors (memvar.h).

```
583 template <typename T>
584 T getHistoryValue(const memvar<T>& mv, const typename memvar<T>::capacityType
      index) {
585   return mv(index);
586 }
587
588 template <typename T, typename Time, typename Clock>
589 T getHistoryValue(const memvarTimed<T, Time, Clock>& mvt, const typename
      memvarTimed<T, Time, Clock>::capacityType index) {
590   return mvt(index);
591 }
592
593 template <typename T, typename Time, typename Clock>
594 T getHistoryTimedValue(const memvarTimed<T, Time, Clock>& mvt, const typename
      memvarTimed<T, Time, Clock>::capacityType index) {
595   return mvt(index);
596 }
```

`mv::getHistoryValue(const memvar<T>&, i)` and `mv::getHistoryTimedValue(const memvarTimed<...>&, i)` both return `T` and are implemented as `mv(i)`. The latter therefore *drops the time tag* and has a different return type than the member of the same name (issue I9).

7 Compilation and Compilation Flags

7.1 Language and library requirements

The header needs a C++20 compiler and standard library. The features it relies on are concepts and constrained templates (`concept`, `requires`, trailing `requires` on non-template members), CRTP, `std::same_as`, `std::remove_cvref_t`, `std::u8string`, the `[[unlikely]]` attribute, `noexcept(constant-expression)`, `if constexpr`, and `std::ranges::minmax_element`. The code was built and run with g++ 13.3 and libstdc++ (the project's CMake file selects g++ and notes that it performed better than `clang++` on the authors' machine, an Ubuntu 24.04). The unit-test file additionally replaces the global `operator new/delete` and silences GCC's `-Wmismatched-new-delete` for that pair (Section 10). `clang++`, other GCC versions, MSVC and other standard libraries were not available for this report.

7.2 Build configuration of the project

The unit-test project (version 1.4.0) uses CMake and sets the following (abridged):

Listing 18: Relevant parts of the project's CMakeLists.txt.

```
PROJECT(memvar-unit-tests VERSION 1.4.0 DESCRIPTION "unit tests for memvar")
CMAKE_MINIMUM_REQUIRED(VERSION 3.26)
SET(CMAKE_C_COMPILER gcc)
SET(CMAKE_CXX_COMPILER g++)
if(NOT DEFINED MEMVAR_SAFEREF_CHECK)
    SET(MEMVAR_SAFEREF_CHECK 2)          # default: MEMVAR_SAFEREF_NO_CHECK
endif()
SET(CMAKE_CXX_STANDARD 20)
SET(CMAKE_CXX_STANDARD_REQUIRED ON)
SET(CMAKE_CXX_EXTENSIONS OFF)
SET(CMAKE_BUILD_TYPE Release)
SET(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -Wextra -pedantic -march=native
    ")   # in the Release branch
FIND_PACKAGE(Threads REQUIRED)
FIND_PACKAGE(GTest REQUIRED)
SET(SOURCE_FILES unitTests.cpp bigint.cpp)
TARGET_COMPILE_DEFINITIONS(${THE_PROJECT} PRIVATE MEMVAR_SAFEREF_CHECK=${
    MEMVAR_SAFEREF_CHECK})
TARGET_LINK_LIBRARIES(${THE_PROJECT} PRIVATE Threads::Threads ${
    GTEST_LIBRARIES} ${GTEST_MAIN_LIBRARIES})
```

Two details of this file are worth knowing: `CMAKE_BUILD_TYPE` is set unconditionally to `Release`, so a `-DCMAKE_BUILD_TYPE=Debug` given on the command line is overridden and the Debug branch (extra warning flags) is never taken; and the macro value is passed on without validation (an invalid value is stopped later by the header's `#error`). GoogleMock headers are included by the tests, but only the GTest libraries are linked, which suffices because no GoogleMock symbol that needs a library is used.

With the Release build type CMake adds `-O3 -DNDEBUG`. The header itself does not use `assert`, so `NDEBUG` has no effect on it. The unit tests are built with:

Listing 19: Building and running the tests in each safety mode.

```
cmake -DMEMVAR_SAFEREF_CHECK=0 -S . -B build && cmake --build build   # abort
    mode
cmake -DMEMVAR_SAFEREF_CHECK=1 -S . -B build && cmake --build build   #
    throwing mode
cmake -DMEMVAR_SAFEREF_CHECK=2 -S . -B build && cmake --build build   #
    default, no check
./build/memvar-unit-tests --gtest_filter=-memVarTest.test_6
```

(`test_6` constructs a history of one billion entries and was excluded on the memory-limited machine used here.)

7.3 Rules that follow from the code

- **Include order for `bigint`.** `memvar.h` only forward declares `bip::bigint`. Printing a `memvar<bip::bigint>` calls `operator«(ostream&, const bip::bigint&)`, which is declared in the *global* namespace in `bigint.h`; it is found only if `bigint.h` is included *before* `memvar.h` (ADL looks in `bip`, not in the global namespace).
- **One value of `MEMVAR_SAFEREF_CHECK` per program.** The macro changes the class layout: the member `writeCount_` and the class `SafeRef` exist only in the checked modes, and `sizeof(memvar<int>)` is 88 bytes in mode 2 and 96 bytes in modes 0 and 1 (96 versus 104 for `memvarTimed<int>`). Translation units compiled with different values violate the one-definition rule (issue I3).
- **Inline everything.** All functions are templates or `inline`, so no library needs to be linked for `memvar.h` itself (the project links `bigint.cpp` for `bigint`).

7.4 Run-time cost

Operation (best of 7, ns per operation)	mode 2	mode 0	mode 1
raw `long long` write (reference)	0.30	0.31	0.30
`memvar<long long>` write, capacity 10	2.40	2.59	2.53
alternating write and read	2.46	2.58	2.54
`m += 1`	2.49	2.52	2.50
`memvarTimed<long long>` write	23.18	24.32	23.43
hand-written deque `push_front`/`pop_back`	2.51	2.49	2.45

Table 7: Micro-benchmark of the delivered header (GCC 13.3, `-O3 -march=native`, 20 M iterations, 5 M for the timed write; virtual machine; differences below about 0.2 ns are within noise).

A write costs about the same as a bare deque `push_front` plus `pop_back`, i.e. the wrapper adds essentially nothing over the deque it wraps, and the safety modes are indistinguishable from the default within the measurement noise (the checked modes are at most about 0.2 ns slower). The timed variant is dominated by the clock read. Nothing is virtual, so there is no indirect call on any path.

Quantity (libstdc++, x86-64)	mode 2	modes 0/1
`sizeof(memvar<int>)` (same for `double`, `std::string`)	88	96
`sizeof(memvarTimed<int>)`	96	104
`sizeof(detail::History<int>)`	88	96
`sizeof(memvarTimed<int>::Entry)`	16	16
`sizeof(detail::SafeRef<int>)`	–	24
`sizeof(std::deque<int>)`	80	80
heap allocations to construct `memvar<int>(0, 10)` (or capacity 1000)	3 (1088 B)	3 (1088 B)
heap allocations to construct `memvarTimed<int>(0, 10)`	3 (1088 B)	3 (1088 B)
heap allocations for 10^6 writes, `memvar<int>`, capacity 10	7812	7812
heap allocations for 10^6 writes, `memvar<long long>`, capacity 10	15625	15625
heap allocations for 10^6 writes, `memvarTimed<int>`, capacity 10	31250	31250

Table 8: Memory footprint and allocation behaviour (the allocation rows are independent of the mode and exclude the three allocations of the construction).

8 Complete Usage Examples

All programs in this section are complete (they compile as shown with `g++ -std=c++20 -I. file.cpp`, adding `bigint.cpp` where `bigint.h` is used), were compiled with `-Wall -Wextra -pedantic` without warnings, and the output printed below each one is the output of that run. Each uses the default safety mode unless stated.

8.1 Basic use: history, indexing, printing, clearing

Listing 20: Basic use of memvar<int>.

```
#include <iostream>
#include "memvar.h"

int main() {
  mv::memvar<int> counter(0, 4);          // initial value 0, history capacity 4

  for (int i = 1; i <= 6; ++i) {
    counter = i * 10;                      // every assignment is recorded
  }
```

```
10
11    std::cout << "current      : " << counter << '\n';
12    std::cout << "size/capacity: " << counter.getHistorySize() << '/'
13              << counter.getHistoryCapacity() << '\n';
14    std::cout << "full         : " << std::boolalpha << counter.isHistoryFull()
          << '\n';
15
16    // index 0 is the newest value, index size-1 the oldest one still stored
17    std::cout << "counter(0)=" << counter(0) << " counter[1]=" << counter[1]
18              << " valueAt(3)=" << counter.valueAt(3) << '\n';
19
20    counter.printHistoryData();             // newest -> oldest
21    counter.printReverseHistoryData(std::cout, ", ");    // oldest -> newest
22
23    const auto [mn, mx] = counter.getHistoryMinMax();
24    std::cout << "min=" << mn << " max=" << mx << '\n';
25
26    const auto [value, error] = counter.getHistoryValue(2);
27    std::cout << "getHistoryValue(2) -> (" << value << ", " << error << ")\n";
28
29    // implicit conversion to T, and the call operator without arguments
30    int plain = counter;
31    std::cout << "plain=" << plain << " counter()=" << counter() << '\n';
32
33    counter.clearHistory();                 // history shrinks to one value: T{}
34    std::cout << "after clear: " << counter << " size=" << counter.
          getHistorySize() << '\n';
35
36    // the free function form
37    counter = 7;
38    counter = 8;
39    std::cout << "free getHistoryValue(counter, 1) = " << mv::getHistoryValue(
          counter, 1) << '\n';
40 }
```

Listing 21: Output of Listing 20.

```
current      : 60
size/capacity: 4/4
full         : true
counter(0)=60 counter[1]=50 valueAt(3)=30
[ 60 50 40 30 ]
[ 30, 40, 50, 60, ]
min=30 max=60
getHistoryValue(2) -> (40, false)
plain=60 counter()=60
after clear: 0 size=1
free getHistoryValue(counter, 1) = 7
```

Points to note: the initial value 0 and the writes 10 and 20 were evicted because the capacity is 4 (so `size/capacity` is `4/4` and `isHistoryFull()` is true); `printReverseHistoryData` lists oldest to newest and also leaves a trailing separator; `clearHistory()` resets the current value to `0`, not to the last value; after clearing, the two following writes give a history in which index 1 is the value 7.

8.2 Operators: arithmetic, comparison, compound assignment

Listing 22: Arithmetic, comparison, compound assignment, increments and strings.

```
1 #include <iostream>
2 #include <string>
3 #include "memvar.h"
```

```cpp

int main() {
  mv::memvar<int> a(5), b(7);

  std::cout << "a+b=" << a + b << " a*3=" << a * 3 << " 100-a=" << 100 - a
            << " b/2=" << b / 2 << '\n';
  std::cout << std::boolalpha
            << "a<b: " << (a < b) << "  a==5: " << (a == 5)
            << "  7==b: " << (7 == b) << "  a!=b: " << (a != b) << '\n';

  a += 10;          // records 15
  a -= b;           // records 8   (memvar operand)
  a *= 2;           // records 16
  a /= 4;           // records 4
  std::cout << "a after compound ops: " << a << '\n';
  a.printHistoryData();

  // prefix returns the new value, postfix the old one; both record a new
      value
  std::cout << "++a=" << ++a << " a++=" << a++ << " now a=" << a << '\n';
  std::cout << "--a=" << --a << " a--=" << a-- << " now a=" << a << '\n';

  // the result of a binary operator is a plain value, not a memvar
  auto sum = a + b;
  static_assert(std::is_same_v<decltype(sum), int>);

  // statements that read and write the same variable are fine
  a = a + a + b;
  std::cout << "a = a + a + b -> " << a << '\n';

  // strings: only + and += (and the comparisons) are available
  mv::memvar<std::string> s("foo", 3);
  s += "bar";
  s = s + "!" ;
  s = "<" + s + ">";
  std::cout << s << " size=" << s.getHistorySize() << '\n';
  s.printHistoryData(std::cout, " | ");
  std::cout << "s == \"<foobar!>\": " << (s == "<foobar!>") << '\n';

  // floating point
  mv::memvar<double> d(1.5);
  d *= 2.0;
  d = d / 4.0;
  std::cout << "d=" << d << " history: ";
  d.printHistoryData();
}
```

Listing 23: Output of Listing 22.

```
a+b=12 a*3=15 100-a=95 b/2=3
a<b: true  a==5: true  7==b: true  a!=b: true
a after compound ops: 4
[ 4 16 8 15 5 ]
++a=5 a++=5 now a=6
--a=5 a--=5 now a=4
a = a + a + b -> 15
<foobar!> size=3
[ <foobar!> | foobar! | foobar | ]
s == "<foobar!>": true
d=0.75 history: [ 0.75 3 1.5 ]
```

The compound operators each record a new value, which is why the history after four of them shows five

entries (`[ 4 16 8 15 5 ]`). The expression `a = a + a + b` reads the current value twice and records the result once. The chain `"++a=" « ++a « " a++=" « a++` is well defined because C++17 sequences the operands of « left to right. The `static_assert` shows that `a + b` has type `int`, not `memvar<int>`.

8.3 Time-tagged variables

Listing 24: memvarTimed with default and custom Time/Clock.

```
#include <chrono>
#include <iostream>
#include <sstream>
#include <thread>
#include "memvar.h"

int main() {
  using namespace std::chrono_literals;

  // default Time = nanoseconds, Clock = steady_clock
  mv::memvarTimed<int> t(100, 3);
  for (int i = 1; i <= 4; ++i) {
    std::this_thread::sleep_for(5ms);
    t = 100 + i;
  }

  std::cout << "value history (newest first): ";
  t.printHistoryData();

  // time tags are offsets from the memvar's own epoch, newest first
  bool monotonic = true;
  for (std::size_t i = 1; i < static_cast<std::size_t>(t.getHistorySize()); ++
      i) {
    monotonic = monotonic && (t.getTimeTag(i - 1) > t.getTimeTag(i));
  }
  std::cout << "tags strictly decreasing with index: " << std::boolalpha <<
      monotonic << '\n';
  std::cout << "newest tag >= 20 ms: " << (t.getTimeTag() >= 20ms) << '\n';

  const auto [v, tag, err] = t.getHistoryTimedValue(1);
  std::cout << "getHistoryTimedValue(1): value=" << v
            << " tag>0: " << (tag.count() > 0) << " err=" << err << '\n';

  // output format of the full timed dump (tags vary from run to run)
  std::ostringstream os;
  t.printHistoryTimedData(os, ";");
  const std::string dump = os.str();
  std::cout << "dump has header: " << (dump.rfind("{ --- begin ---\n[TimeTag:
      Value]\n", 0) == 0)
            << ", footer: " << dump.ends_with("  --- end --- }\n\n") << '\n';

  // custom resolution and clock
  mv::memvarTimed<double, std::chrono::milliseconds, std::chrono::system_clock
      > w(1.0, 5);
  std::this_thread::sleep_for(12ms);
  w = 2.0;
  std::cout << "ms tag >= 12ms: " << (w.getTimeTag() >= 12ms) << '\n';

  // floating-point duration
  using fsec = std::chrono::duration<double>;
  mv::memvarTimed<int, fsec> f(1);
  f = 2;
  std::cout << "fsec tag is double: "
            << std::is_same_v<decltype(f.getTimeTag().count()), double> << '\n
                ';
```

```

  // clearHistory() restarts the epoch: the single remaining value has tag 0
  t.clearHistory();
  std::cout << "after clear: value=" << t << " size=" << t.getHistorySize() <<
       '\n';

  // streaming a timed memvar prints "[newest tag:value]"
  std::ostringstream one;
  one << t;
  std::cout << "streamed form begins with '[': " << (one.str().front() == '[')
       << '\n';
}
```

Listing 25: Output of Listing 24.

```
value history (newest first): [ 104 103 102 ]
tags strictly decreasing with index: true
newest tag >= 20 ms: true
getHistoryTimedValue(1): value=103 tag>0: true err=false
dump has header: true, footer: true
ms tag >= 12ms: true
fsec tag is double: true
after clear: value=[0:0] size=1
streamed form begins with '[': true
```

The actual tag values depend on the run, so the program checks properties instead of printing them: tags decrease with the index, the newest tag is at least the sum of the sleeps, and the dump has the documented header and footer. The third object uses `duration<double>` as `Time`. The last lines show that `clearHistory()` leaves one value with tag zero, and that streaming a timed variable prints the tag/value pair, here `[0:0]`.

8.4 Big integers: a Fibonacci recurrence

Listing 26: memvar<bip::bigint> with bigint.h included first.

```
#include <iostream>
#include "bigint.h"     // must come BEFORE memvar.h (see Section on
    constraints)
#include "memvar.h"

int main() {
  // Fibonacci with a memory of the previous two terms only
  mv::memvar<bip::bigint> fib(bip::bigint(1), 2);
  fib = bip::bigint(1);                          // fib(0) = 1, fib(1) = 1
  for (int i = 3; i <= 100; ++i) {
    fib += fib(1);                               // new = current + previous
  }
  std::cout << "F(100) = " << fib << '\n';

  // mixed bigint / int32_t arithmetic is forwarded to bigint's own operators
  mv::memvar<bip::bigint> big(bip::bigint("10000000000000000000"), 4);
  big = big * 3;
  big += 7;
  std::cout << "big = " << big << '\n';
  std::cout << "big > 5: " << std::boolalpha << (big > 5) << '\n';
  std::cout << "history size = " << big.getHistorySize() << '\n';

  try {
    big /= bip::bigint(0);                       // bigint throws; nothing is
        recorded
  } catch (const std::invalid_argument& e) {
```

```
    std::cout << "caught: " << e.what() << ", history size still "
              << big.getHistorySize() << '\n';
  }
}
```

Listing 27: Output of Listing 26.

```
F(100) = 354224848179261915075
big = 300000000000000000007
big > 5: true
history size = 3
caught: Division by zero, history size still 3
```

A capacity of 2 is the minimum and is exactly what a two-term recurrence needs: after `fib = 1` the history is `[1, 1]`, and `fib += fib(1)` computes "current plus previous" and records it, which evicts the older of the two. The failed division demonstrates that the arithmetic in a compound assignment is evaluated *before* anything is recorded: the history size is still 3 after the exception.

8.5 A realistic pattern: a sensor filter

Listing 28: Moving average and jump detection on top of getMemVarHistory().

```
#include <iostream>
#include <numeric>
#include "memvar.h"

// A small sensor filter: keep the last 5 samples, expose a moving average
// and a "changed more than a threshold since the previous sample" test.
class Sensor {
public:
  void sample(int raw) { raw_ = raw; }
  double movingAverage() const {
    const auto& h = raw_.getMemVarHistory();            // const std::deque<
        int>&
    return std::accumulate(h.begin(), h.end(), 0.0) / static_cast<double>(h.
        size());
  }
  bool jumped(int threshold) const {
    if (raw_.getHistorySize() < 2) return false;
    const int delta = raw_(0) - raw_(1);                // newest - previous
    return (delta < 0 ? -delta : delta) > threshold;
  }
  int last() const { return raw_(); }
private:
  mv::memvar<int> raw_ {0, 5};
};

int main() {
  Sensor s;
  for (int v : {10, 11, 9, 10, 40, 41, 42}) {
    s.sample(v);
    std::cout << "sample " << v << "  avg(last 5) = " << s.movingAverage()
              << (s.jumped(15) ? "   <-- jump" : "") << '\n';
  }
}
```

Listing 29: Output of Listing 28.

```
sample 10  avg(last 5) = 5
sample 11  avg(last 5) = 7
```

```
sample 9  avg(last 5) = 7.5
sample 10  avg(last 5) = 8
sample 40  avg(last 5) = 16   <-- jump
sample 41  avg(last 5) = 22.2
sample 42  avg(last 5) = 28.4
```

This is the intended style of use: the owner of the variable assigns raw samples, and the history provides both the moving window (iterated without copying through `getMemVarHistory()`) and the previous sample (`raw_(1)`). The first averages are low because the window initially holds the constructor's `0`.

8.6 Dangling-reference protection in the three modes

Listing 30: Using cref() and observing the effect of MEMVAR_SAFEREF_CHECK.

```
#include <iostream>
#include "memvar.h"

int main() {
  mv::memvar<int> m(1, 3);                          // capacity 3
  auto h = m.cref();                                // handle on the value 1
  m = 2;
  m = 3;
  std::cout << "after 2 more writes: " << h << '\n';   // still alive (3
      values stored)

#if MEMVAR_SAFEREF_CHECK == MEMVAR_SAFEREF_THROWS
  m = 4;                                            // evicts the slot h refers
      to
  try {
    std::cout << h << '\n';
  } catch (const mv::expired_history_reference& e) {
    std::cout << "expired handle detected: " << e.what() << '\n';
  }
#elif MEMVAR_SAFEREF_CHECK == MEMVAR_SAFEREF_ABORT
  m = 4;                                            // evicts the slot h refers
      to
  std::cout << "dereferencing the expired handle now aborts the process\n";
  std::cout.flush();
  std::cout << h << '\n';                           // prints FATAL... and abort
      ()s
#else
  std::cout << "NO_CHECK: cref() is just const int&, "
            << "type is int const&: "
            << std::is_same_v<decltype(m.cref()), const int&> << '\n';
#endif
}
```

The same source gives three different behaviours, selected only by the macro.

Listing 31: Compile with -DMEMVAR_SAFEREF_CHECK=0 (abort mode); the process ends with signal 6.

```
after 2 more writes: 1
dereferencing the expired handle now aborts the process
FATAL: mv::memvar::cref(): the referenced value has been evicted from history
    (too many writes, or a clearHistory(), happened since cref() was called).
    Terminating.
```

Listing 32: Compile with -DMEMVAR_SAFEREF_CHECK=1 (throwing mode).

```
after 2 more writes: 1
```

```
expired handle detected: mv::memvar::cref(): the referenced value has been
    evicted from history (too many writes, or a clearHistory(), happened since
     cref() was called)
```

Listing 33: Default build (-DMEMVAR_SAFEREF_CHECK=2): no handle exists.

```
after 2 more writes: 1
NO_CHECK: cref() is just const int&, type is int const&: 1
```

In modes 0 and 1 the handle is valid after two further writes (the history of capacity 3 still holds the value) and expires on the third, exactly as in Table 4. In mode 2 there is nothing to expire, and code that keeps the reference across three writes would be reading a destroyed element.

8.7 Strings and Unicode output

Listing 34: Printing wide and Unicode strings; output shown as UTF-8 bytes.

```
#include <iomanip>
#include <iostream>
#include <sstream>
#include "memvar.h"

static void hex(const std::string& s) {
  for (unsigned char c : s) std::cout << std::hex << std::setw(2) << std::
      setfill('0') << int(c) << ' ';
  std::cout << std::dec << "(" << s.size() << " bytes)\n";
}

int main() {
  // UTF-32 -> UTF-8: U+00E9 and U+1F600
  mv::memvar<std::u32string> u32(U"\u00e9\U0001F600");
  std::ostringstream a; a << u32;  hex(a.str());

  // UTF-16 surrogate pair is combined into one 4-byte UTF-8 sequence
  mv::memvar<std::u16string> u16(u"\U0001F600");
  std::ostringstream b; b << u16;  hex(b.str());

  // a lone surrogate becomes U+FFFD
  mv::memvar<std::u16string> bad(std::u16string(1, char16_t(0xD800)));
  std::ostringstream c; c << bad;  hex(c.str());

  // wstring on this platform (wchar_t is 32 bits on Linux)
  mv::memvar<std::wstring> w(L"\u20ac");
  std::ostringstream d; d << w;    hex(d.str());

  // u8string is forwarded byte for byte
  mv::memvar<std::u8string> u8(u8"\u00e9");
  std::ostringstream e; e << u8;   hex(e.str());

  // string history
  mv::memvar<std::string> s("a", 3);
  s = "b"; s = "c"; s = "d";
  s.printReverseHistoryData(std::cout, ",");
}
```

Listing 35: Output of Listing 34.

```
c3 a9 f0 9f 98 80 (6 bytes)
f0 9f 98 80 (4 bytes)
ef bf bd (3 bytes)
e2 82 ac (3 bytes)
```

```
c3 a9 (2 bytes)
[ b,c,d,]
```

The first line is U+00E9 followed by U+1F600 encoded as `c3 a9` and `f0 9f 98 80`; the UTF-16 surrogate pair becomes the same four bytes as one code point; a lone high surrogate becomes the replacement character `ef bf bd`; and `std::u8string` bytes pass through unchanged.

9 Edge Cases

The program below collects the compile-time properties (checked with `static_assert`, so a regression would break the build) and the run-time edge cases in one place.

Listing 36: Edge-case program: compile-time properties and run-time behaviour.

```
#include <cstdint>
#include <iostream>
#include <string>
#include <type_traits>
#include <vector>
#include "bigint.h"
#include "memvar.h"

using mv::memvar;
using mv::memvarTimed;

// ---- compile-time properties (all checked with static_assert) -------------
static_assert(!std::is_copy_constructible_v<memvar<int>>);
static_assert(!std::is_move_constructible_v<memvar<int>>);
static_assert(!std::is_move_assignable_v<memvar<int>>);            // deleted overload wins
static_assert(std::is_copy_assignable_v<memvar<int>>);             // value copy only
static_assert(!std::is_copy_constructible_v<memvarTimed<int>>);
static_assert(!std::is_move_constructible_v<memvarTimed<int>>);
static_assert(!std::is_move_assignable_v<memvarTimed<int>>);
static_assert(!std::is_polymorphic_v<memvar<int>>);                // nothing virtual
static_assert(!std::is_polymorphic_v<memvarTimed<int>>);
// the two classes are unrelated types
static_assert(!std::is_base_of_v<memvar<int>, memvarTimed<int>>);
static_assert(!std::is_convertible_v<memvarTimed<int>&, const memvar<int>&>);
// both satisfy IsMemvar; arbitrary types do not
static_assert( mv::IsMemvar<memvar<int>> && mv::IsMemvar<memvarTimed<int>>);
static_assert(!mv::IsMemvar<int> && !mv::IsMemvar<std::string>);
// assignment returns the derived type
static_assert(std::is_same_v<decltype(std::declval<memvar<int>&>() = 1), memvar<int>&>);
static_assert(std::is_same_v<decltype(std::declval<memvarTimed<int>&>() = 1), memvarTimed<int>&>);
// the value constructor is explicit
static_assert(!std::is_convertible_v<int, memvar<int>>);
static_assert(std::is_constructible_v<memvar<int>, int>);

template <class L, class R> concept HasPlusEq  = requires(L& l, const R& r) { l += r; };
template <class L, class R> concept HasMinusEq = requires(L& l, const R& r) { l -= r; };
template <class L, class R> concept HasMinus   = requires(const L& l, const R& r) { l - r; };
template <class L>          concept HasPreInc  = requires(L& l) { ++l; };
template <class L, class R> concept HasMod     = requires(const L& l, const R&
```

```
    r) { l % r; };
template <class L>          concept HasNeg     = requires(const L& l) { -l; };

static_assert( HasPlusEq <memvar<std::string>, const char*>);
static_assert(!HasMinusEq<memvar<std::string>, std::string>);
static_assert(!HasMinus  <memvar<std::string>, std::string>);
static_assert(!HasPreInc <memvar<std::string>>);
static_assert( HasPreInc <memvar<double>>);
static_assert( HasMod    <memvar<int>, int>);            // via implicit conversion to int
static_assert(!HasMod    <memvar<bip::bigint>, int>);    // member operator: no conversion
static_assert( HasNeg    <memvar<int>>);                 // via implicit conversion
static_assert(!HasNeg    <memvar<bip::bigint>>);

template <class T> concept Constructible = requires { typename memvar<T>; };

// result types of the binary operators (no re-narrowing to T)
static_assert(std::is_same_v<decltype(std::declval<memvar<std::int8_t>&>() + std::declval<memvar<std::int8_t>&>()), int>);
static_assert(std::is_same_v<decltype(std::declval<memvar<int>&>() + 1.5), double>);
static_assert(std::is_same_v<decltype(std::declval<memvar<int>&>() == std::declval<memvar<long>&>()), bool>);

template <class F> void attempt(const char* what, F f) {
  try { f(); std::cout << what << ": no exception\n"; }
  catch (const std::exception& e) { std::cout << what << ": " << typeid(e).name() << " -> " << e.what() << '\n'; }
}

int main() {
  attempt("capacity 1", [] { memvar<int> m(0, 1); });
  attempt("capacity 0", [] { memvar<int> m(0, 0); });
  attempt("capacity -5", [] { memvar<int> m(0, -5); });
  attempt("capacity 2 ", [] { memvar<int> m(0, 2); });

  memvar<int> m(10, 3);
  attempt("index -1", [&] { (void)m(-1); });
  attempt("index 1 (size 1)", [&] { (void)m.valueAt(1); });
  attempt("getHistoryValue(5)", [&] { (void)m.getHistoryValue(5); });
  memvarTimed<int> t(1, 3);
  attempt("timed getTimeTag(3)", [&] { (void)t.getTimeTag(3); });
  attempt("timed getTimeTag(-1)", [&] { (void)t.getTimeTag(-1); });
  attempt("timed getHistoryTimedValue(-1)", [&] { (void)t.getHistoryTimedValue(-1); });

  // self assignment records a duplicate
  m = m;
  std::cout << "after self-assign size=" << m.getHistorySize() << " : ";
  m.printHistoryData();

  // compound assignment with a different arithmetic type truncates silently
  memvar<int> q(1, 4);
  q += 1.9;  std::cout << "int += 1.9  -> " << q << '\n';
  memvar<std::int8_t> small(120, 3);
  small += 20; std::cout << "int8 120+=20 -> " << int(small()) << '\n';

  // ++ on a narrow type is computed in int and converted back
  memvar<unsigned char> uc(255, 3);
  uc++;        std::cout << "uchar 255++ -> " << int(uc()) << '\n';

```

```cpp
  // char-like T are printed as characters
  memvar<char> ch('A', 3);  ++ch;
  std::cout << "char: " << ch << '\n';

  // clearHistory resets to T{}, not to the latest value
  memvar<int> keep(42, 3); keep = 43; keep.clearHistory();
  std::cout << "after clearHistory: " << keep << '\n';

  // postfix returns the old value
  memvar<int> p(5, 3);
  int old = p++;
  std::cout << "p++ returned " << old << ", p is " << p << '\n';

  // auto cref: in NO_CHECK mode this is a COPY of the value (auto drops the reference)
  auto c = p.cref();
  std::cout << "auto cref is a handle type: " << !std::is_same_v<decltype(c), int> << '\n';

  // mixed comparisons between different memvar types
  memvar<long> l(7);
  memvar<int> i7(7);
  std::cout << "memvar<int>(7) == memvar<long>(7): " << (i7 == l) << '\n';
  memvarTimed<int> ti(7);
  std::cout << "memvar<int> == memvarTimed<int>: " << (i7 == ti) << '\n';

  // bare memvar assigned to a timed one: goes through operator T()
  ti = i7;
  ti += i7;
  std::cout << "timed after = and += from bare memvar: " << ti() << '\n';
}
```

Listing 37: Output of Listing 36 (default mode).

```
capacity 1: St16invalid_argument -> ERROR: The history capacity must be 2 at
    least, or more
capacity 0: St16invalid_argument -> ERROR: The history capacity must be 2 at
    least, or more
capacity -5: St16invalid_argument -> ERROR: The history capacity must be 2 at
    least, or more
capacity 2 : no exception
index -1: St12out_of_range -> Index -1 is out of bounds (size is 1).
index 1 (size 1): St12out_of_range -> Index 1 is out of bounds (size is 1).
getHistoryValue(5): St12out_of_range -> Index 5 is out of bounds (size is 1).
timed getTimeTag(3): St12out_of_range -> Index 3 is out of bounds (size is 1).
timed getTimeTag(-1): St12out_of_range -> Index -1 is out of bounds (size is
    1).
timed getHistoryTimedValue(-1): St12out_of_range -> Index -1 is out of bounds
    (size is 1).
after self-assign size=2 : [ 10 10 ]
int += 1.9  -> 2
int8 120+=20 -> -116
uchar 255++ -> 0
char: B
after clearHistory: 0
p++ returned 5, p is 6
auto cref is a handle type: 0
memvar<int>(7) == memvar<long>(7): 1
memvar<int> == memvarTimed<int>: 1
timed after = and += from bare memvar: 14
```

9.1 Capacity and index errors

- Capacities below 2 (including 0 and negative values) throw `std::invalid_argument` from the constructor, before any storage exists. Capacity 2 is accepted. The minimum is a fixed constant (`detail::minimumHistoryCapacity`).
- A value access with an index that is negative or not smaller than the current *size* (not the capacity) throws `std::out_of_range` with the message `Index` *i* `is out of bounds (size is` *n*`).`: a freshly constructed variable has size 1, so `m(1)` throws until a second value has been written.
- The time-tag accessors `getTimeTag` and `getHistoryTimedValue` report the same condition with the same message, including for negative indices (the run above shows `getTimeTag(3)`, `getTimeTag(-1)` and `getHistoryTimedValue(-1)`).
- The capacity is converted to `int64_t` by an ordinary function call, so a floating-point capacity is truncated silently *(by inspection)*.

9.2 Assignment edge cases

- **Self-assignment** (`m = m`) is not a no-op: it records the current value again (history `[ 10 10 ]`), pushing older values out. Since the deque keeps references to existing elements valid when an element is added at the front, constructing the new element from the old front element is safe.
- **Assigning from a bare `memvar` to a `memvarTimed`** and `+=` with a bare `memvar` on the right both work (they go through the conversion to `T` and through the generic operator, respectively): the program prints 14 after `ti = i7; ti += i7;` with `i7 == 7`.
- **Mixed value types** compare and add (`memvar<int> == memvar<long>`, `memvar<int> == memvarTimed<int>`), because the operator templates only require that the underlying operation exist.
- **Chained assignment** works because every assignment returns the derived object (`a = b = 5` in Listing 42).
- **A const variable on the left of a compound assignment** satisfies the constraint (`IsMemvar<const memvar<int»` is true, and a `static_assert` that `HasPlusEq<const memvar<int>, int>` holds compiles), but the body then fails to compile:

Listing 38: Compound assignment on a const variable (g++ 13.3).

```
const mv::memvar<int> c(1);  c += 1;
memvar.h:624:22: error: no matching function for call to 'mv::memvarAccess::
    set(const mv::memvar<int>&, int)'
```

9.3 Numeric edge cases

- **Silent truncation in compound assignment:** adding 1.9 to a `memvar<int>` holding 1 records 2; adding 20 to a `memvar<int8_t>` holding 120 records −116 (the sum is computed in `int` and converted back).
- **Narrow types and `++`:** `memvar<unsigned char>(255)++` records 0 (computed in `int`, converted back with `static_cast`); `memvar<char>('A')` increments to `'B'` and prints as a character.
- **Signed overflow and division by zero** for `int`-width types and wider are those of the built-in operators: `INT_MAX + 1` or `m /= 0` is undefined behaviour *(by inspection; not run)*. Only `bigint` reports division by zero as an exception (`std::invalid_argument`, seen in Listing 26).

- **Mixed signedness:** the operators forward to the built-in comparison, so `memvar<int>(-1) < memvar<unsigned>(1)` yields `false`, exactly like `-1 < 1u`. Since the operator body is the plain expression `valueOf(lhs) < valueOf(rhs)` inside `memvar.h`, GCC does warn (`-Wsign-compare`); the warning points at the macro invocation line in the header, and the user's own line appears only as "required from here":

Listing 39: Output and compiler diagnostic of the mixed-signedness probe.

```
memvar<int>(-1) < memvar<unsigned>(1): false
-1 < 1u (built-in): false
--- compiler diagnostic (g++ 13.3 -Wall -Wextra):
edge6_mixed.cpp:5:84:   required from here
memvar.h:609:24: warning: comparison of integer expressions of different
    signedness: 'const int' and 'const unsigned int' [-Wsign-compare]
edge6_mixed.cpp:6:46: warning: comparison of integer expressions of different
    signedness: 'int' and 'unsigned int' [-Wsign-compare]
```

9.4 Reference-handle edge cases

- In the default mode `auto c = m.cref();` deduces `int` and copies; the edge-case program prints `0` for "is a handle type". In modes 0 and 1 the same statement declares a `SafeRef` and prints `1`.
- `clearHistory()` expires every outstanding handle even though only a single write is logically involved.
- A handle is a view of an *entry*: `h = m.cref(); m = 99;` leaves `h` designating the previous value.
- A write that throws does not count towards the expiry of a handle (modes 0 and 1; covered by the unit test `DoesNotAgeOutstandingSafeRefs`).
- **Very large capacities.** The expiry value is computed as `writeCount + capacity`, and `reset` adds the capacity to the counter. With a capacity near the maximum of `int64_t` this is signed overflow in the checked modes: a program built with mode 1 and `-fsanitize=undefined` that constructs `memvar<int>(0, INT64_MAX)`, writes once and calls `cref()` stops with `memvar.h:208:43: runtime error: signed integer overflow: 9223372036854775807 + 2 cannot be represented in type 'long int'` (reproduced). The default mode has no such arithmetic.

9.5 Failed writes: the strong guarantee

The program below makes a write fail deterministically, without limiting the address space of the process: the global `operator new` is replaced by a version that throws `std::bad_alloc` for the next allocation of at least 1 000 bytes once armed (the 5 001-byte string copy; the deque nodes are far smaller). It then checks that a `memvarTimed<std::string>` is exactly as it was.

Listing 40: A failed write on a memvarTimed changes nothing.

```
// A failed write (here: std::bad_alloc while copying a large string) must
    leave a
// memvarTimed unchanged: same size, same value, same tags.
// No ulimit needed: a replaced global operator new fails on demand.
#include <cstdio>
#include <cstdlib>
#include <iostream>
#include <new>
#include <string>
#include "memvar.h"

static bool g_failNextBigAlloc = false;           // armed just before the
    write
```

```cpp
void* operator new(std::size_t n) {
  if (g_failNextBigAlloc && n >= 1000) {          // only the string copy is
      that large
    g_failNextBigAlloc = false;
    throw std::bad_alloc();
  }
  if (void* p = std::malloc(n)) return p;
  throw std::bad_alloc();
}
void operator delete(void* p) noexcept { std::free(p); }
void operator delete(void* p, std::size_t) noexcept { std::free(p); }

int main() {
  mv::memvarTimed<std::string> m("first", 10);
  const std::string big(5000, 'x');               // allocated BEFORE arming
      the failure

  std::cout << "size before failed write : " << m.getHistorySize() << '\n';

  g_failNextBigAlloc = true;
  try {
    m = big;                                      // copy -> allocation ->
        bad_alloc
    std::cout << "write unexpectedly succeeded\n";
  } catch (const std::bad_alloc&) {
    std::cout << "write failed with bad_alloc\n";
  }
  g_failNextBigAlloc = false;

  std::cout << "size after failed write  : " << m.getHistorySize() << '\n';

  // With a consistent object, getTimeTag(1) must throw: only index 0 exists.
  try {
    (void)m.getTimeTag(1);
    std::cout << "BUG: getTimeTag(1) succeeded, time history is longer than
        value history\n";
  } catch (const std::out_of_range&) {
    std::cout << "getTimeTag(1) threw (consistent)\n";
  }

  m = "second";                                   // a normal write
  std::cout << "after next good write: value(0)=" << m(0) << " value(1)=" << m
      (1) << '\n';
  try {
    (void)m.getTimeTag(2);
    std::cout << "BUG: getTimeTag(2) succeeded but the value history has only
        "
              << m.getHistorySize() << " entries\n";
  } catch (const std::out_of_range&) {
    std::cout << "getTimeTag(2) threw (consistent: " << m.getHistorySize() <<
        " entries)\n";
  }
}
```

Listing 41: Output of Listing 40.

```
size before failed write : 1
write failed with bad_alloc
size after failed write  : 1
getTimeTag(1) threw (consistent)
after next good write: value(0)=second value(1)=first
getTimeTag(2) threw (consistent: 2 entries)
```

After the failed write the size is unchanged, `getTimeTag(1)` throws (only index 0 exists), and after the next good write the entry of `"first"` still carries its own tag, because the value and the tag of an entry are one object inserted by one `emplace_front`. The unit tests of Section 10 turn this program into assertions for plain and timed variables, for full histories and for `clearHistory()`.

9.6 Consequences of the type structure

The following program checks, partly at compile time, the properties that follow from the CRTP structure: `memvarTimed` is not accepted where `const memvar<int>&` is expected, but is accepted by a function template constrained with `mv::IsMemvar`; the history container of a timed variable holds `TimedEntry` objects; the mixin cannot be destroyed through its own type; the member `getHistoryTimedValue` keeps the tag while the free function of the same name returns the value only; and `clearHistory` moves the epoch.

Listing 42: Edge cases of the type structure.

```
// Edge cases of the type structure: two unrelated classes, one shared interface.
#include <chrono>
#include <iostream>
#include <string>
#include <thread>
#include <type_traits>
#include "memvar.h"

using namespace std::chrono_literals;
using mv::memvar;
using mv::memvarTimed;

// written against the plain class ...
int previous(const memvar<int>& m) { return m(1); }
// ... and generic code that takes either flavour
template <mv::IsMemvar M> int previousAny(const M& m) { return m(1); }

template <class X> concept AcceptedByPlainRef = requires(X& x) { previous(x); };

using Clk = std::chrono::steady_clock;
static_assert( AcceptedByPlainRef<memvar<int>>);
static_assert(!AcceptedByPlainRef<memvarTimed<int>>);
static_assert(std::is_same_v<decltype(std::declval<const memvar<int>&>().getMemVarHistory()),
                             const std::deque<int>&>);
static_assert(std::is_same_v<decltype(std::declval<const memvarTimed<int>&>().getMemVarHistory()),
                             const std::deque<memvarTimed<int>::Entry>&>);
static_assert(std::is_same_v<memvarTimed<int>::Entry, mv::detail::TimedEntry<int, Clk::time_point>>);
static_assert(std::is_same_v<memvarTimed<int>::time_point, Clk::time_point>);
static_assert(std::is_same_v<decltype(mv::getHistoryTimedValue(std::declval<const memvarTimed<int>&>(), 0)), int>);
static_assert(std::is_same_v<decltype(std::declval<const memvarTimed<int>&>().getHistoryTimedValue(0)),
                             std::tuple<int, std::chrono::nanoseconds, bool>>);
// the shared interface is a CRTP mixin: both classes derive from their own instantiation
static_assert(std::is_base_of_v<mv::detail::ValueInterface<memvar<int>, int, int>, memvar<int>>);
static_assert(std::is_base_of_v<mv::detail::ValueInterface<memvarTimed<int>, int, memvarTimed<int>::Entry>, memvarTimed<int>>);
```

```cpp
// the mixin's destructor is protected: it cannot be destroyed through a base pointer
static_assert(!std::is_destructible_v<mv::detail::ValueInterface<memvar<int>, int, int>>);
// the handle type depends on the build mode
#if MEMVAR_SAFEREF_CHECK == MEMVAR_SAFEREF_NO_CHECK
static_assert(std::is_same_v<decltype(std::declval<const memvar<int>&>().cref()), const int&>);
#else
static_assert(std::is_same_v<decltype(std::declval<const memvar<int>&>().cref()), memvar<int>::SafeRef>);
#endif

int main() {
  std::cout << "sizeof(memvarTimed<int>::Entry) = " << sizeof(memvarTimed<int>::Entry) << '\n';

  memvarTimed<int> t(10, 4);
  std::this_thread::sleep_for(2ms);
  t = 20;
  std::this_thread::sleep_for(2ms);
  t = 30;

  std::cout << "previousAny(t)                    = " << previousAny(t) << '\n';

  // the history container of a timed variable holds {value, tag} pairs
  std::cout << "entries (value, tag >= 2 ms after the oldest tag):";
  const auto oldest = t.getMemVarHistory().back().tag;
  for (const auto& e : t.getMemVarHistory())
    std::cout << " (" << e.value << ", " << (e.tag - oldest >= 2ms) << ")";
  std::cout << '\n';

  std::cout << "first value has tag exactly zero: " << (t.getTimeTag(2) == std::chrono::nanoseconds::zero()) << '\n';

  // the free function drops the tag, the member keeps it
  auto [v, tag, err] = t.getHistoryTimedValue(1);
  std::cout << "member getHistoryTimedValue(1)  -> value " << v << ", tag>=2ms: "
            << (tag >= 2ms) << ", err " << err << '\n';
  std::cout << "free   getHistoryTimedValue(t,1) -> " << mv::
      getHistoryTimedValue(t, 1) << " (value only)\n";

  // assignment returns the derived object, so assignments chain
  memvarTimed<int> a(0, 3), b(0, 3);
  a = b = 5;
  std::cout << "chained assignment a = b = 5    -> a=" << a() << " b=" << b() << '\n';

  // clearHistory moves the epoch to the new single entry
  t.clearHistory();
  std::cout << "after clearHistory: size " << t.getHistorySize()
            << ", getTimeTag(0) = " << t.getTimeTag(0).count() << " ns, value "
                " << t() << '\n';

  // min/max works on the value part only
  memvarTimed<int> r(5, 4); r = 1; r = 9;
  auto [mn, mx] = r.getHistoryMinMax();
  std::cout << "getHistoryMinMax -> " << mn << " " << mx << '\n';
}
```

Listing 43: Output of Listing 42.

```
sizeof(memvarTimed<int>::Entry) = 16
previousAny(t)                  = 20
entries (value, tag >= 2 ms after the oldest tag): (30, 1) (20, 1) (10, 0)
first value has tag exactly zero: 1
member getHistoryTimedValue(1)  -> value 20, tag>=2ms: 1, err 0
free   getHistoryTimedValue(t,1) -> 20 (value only)
chained assignment a = b = 5    -> a=5 b=5
after clearHistory: size 1, getTimeTag(0) = 0 ns, value 0
getHistoryMinMax -> 1 9
```

9.7 Stream operator lookup

Listing 44: Name lookup of operator« inside a namespace that declares its own.

```
#include <iostream>
#include <sstream>
#include "memvar.h"
namespace app {
struct Point { int x; };
std::ostream& operator<<(std::ostream& os, const Point& p) { return os << p.x; }
std::string viaApp(const mv::memvarTimed<int>& m) { std::ostringstream o; o << m; return o.str(); }
std::string viaAppW([[maybe_unused]] const mv::memvar<std::wstring>& m) { std::ostringstream o;
#ifdef TRY_W
  o << m;
#endif
  return o.str(); }
}
int main() {
  mv::memvarTimed<int> m(3);
  std::ostringstream o; o << m;
  std::cout << "global scope : " << o.str() << '\n';
  std::cout << "inside app   : " << app::viaApp(m) << '\n';
}
```

Listing 45: Output of Listing 44 (without TRY_W).

```
global scope : [0:3]
inside app   : 3
```

Inside namespace `app`, which declares an `operator«` for an unrelated type, the global `operator«` for `memvarTimed` is hidden. The expression still compiles for an `int` payload because the implicit `operator T()` converts the object to `int`: the output silently changes from `[0:3]` to `3`. With `-DTRY_W`, the same statement for a `memvar<std::wstring>` does not compile ("no match for `operator«`"), because there is no `std::wstring` insertion visible from that scope either.

9.8 Unusable but accepted types

As Table 2 showed, `const T` and reference types satisfy the concept but fail inside the standard library or in the mixin. `bool` and `std::string_view` are rejected cleanly ("template constraint failure").

9.9 Containers and storage of memvars

Because copy and move construction are deleted, a `memvar` cannot be returned by value from a function that does not elide it, cannot be an element of a `std::vector` whose storage may be reallocated, and has no `swap`. Node-based containers that construct elements in place (`std::list::emplace_back`,

`std::deque::emplace_back`, `std::map::try_emplace`) and `std::unique_ptr<memvar<T>>` (used by the unit tests) are the usable options.

10 The Unit-Test Suite

10.1 Structure and results

The project's `unitTests.cpp` (1 531 lines) uses GoogleTest; it includes `bigint.h` before `memvar.h`, as required by the include-order rule of Section 7, and defines 49 `TEST`s in seven suites (Table 9). Most tests are written in a direct style: perform a sequence of writes, then assert the indexed values, sizes, capacities and comparison results; many also print the history to the standard streams, but no test captures or checks printed text.

Suite	Tests	What it covers
`memVarTimedTest`	13	the timed variable: ordered writes and indexing, constructor validation for `int`, `double`, `string`, `bigint`, capacity 2 with `getHistoryMinMax`, default construction, increments and compound operators, a full history and `clearHistory`, strings, `char` with `++`/`-`, comparisons between timed variables, the tuple accessors, timing loops, Fibonacci with `int64_t` and `bigint`
`memVarTest`	17	the same list for the plain variable, plus `crefTest` (three `#if` branches, one per safety mode, including `EXPECT_THROW` and `EXPECT_DEATH`), passing by reference, `make_unique`/`make_shared` owners, the `%` operator through the conversion, and two stress tests (`test_6`: a history of 10^9 entries; `test_7`: 10^9 writes with capacity 100)
`MemvarTimedAssignFromBare Memvar`	3	`timed = plain` compiles and gives the right value, also for `std::string`
`MemvarTimedCompoundAssign FromBareMemvar`	6	`timed += plain` etc. are well-formed (checked with `requires`-expression concepts) and give the right values
`MemvarFailedWrite`	5	failed writes on plain variables
`MemvarTimedFailedWrite`	3	failed writes on timed variables
`MemvarTimedEntries`	2	values and tags are paired; tag index checking

Table 9: The seven test suites.

The tests were built and run in each safety mode (`-O3 -DNDEBUG -Wall -Wextra -pedantic -march=native`); `memVarTest.test_6` was excluded in every run, because it allocates about 8 GB.

Mode	Defined	Run	Passed	Remark
0 (abort)	49	48	48	also under ASan+UBSan
1 (throws)	49	48	48	also under ASan+UBSan
2 (no check)	48	47	47	`DoesNotAgeOutstandingSafeRefs` is compiled out; also under ASan+UBSan

Table 10: Unit-test results against the delivered header.

The sanitizer runs used the following options, with AddressSanitizer's allocation/deallocation-mismatch check switched off (`ASAN_OPTIONS=alloc_dealloc_mismatch=0`), because the test file replaces `operator new` and `operator delete` with `malloc`/`free`-based versions. No report was produced (each run took about 80 s, almost all of it `test_7`).

Listing 46: Sanitizer build options.

```
-O1 -g -fsanitize=address,undefined -fno-sanitize-recover=undefined
```

10.2 Injecting an allocation failure

A failed write cannot be provoked portably with real memory pressure, so the tests replace the global allocation function. The relevant part of the file is shown in Listing 47.

Listing 47: Failure injection in unitTests.cpp.

```
1309 #include <cstdlib>
1310 #include <new>
1311
1312 namespace failinj {
1313 inline bool armed = false;
1314 inline constexpr std::size_t threshold = 1000;
1315 }  // namespace failinj
1316
1317 // The replacement pair is malloc/free based, which is valid; but at -O2/-O3 GCC
1318 // inlines operator new/delete into callers (e.g. std::make_unique) and then
1319 // reports -Wmismatched-new-delete because it sees free() applied to a pointer
1320 // that came from operator new. The warning is a false positive for a matched
1321 // replacement pair, so it is silenced for these definitions only.
1322 #if defined(__GNUC__) && !defined(__clang__)
1323 #pragma GCC diagnostic push
1324 #pragma GCC diagnostic ignored "-Wmismatched-new-delete"
1325 #endif
1326 void* operator new(std::size_t n) {
1327   if (failinj::armed && n >= failinj::threshold) {
1328     failinj::armed = false;               // fail exactly once
1329     throw std::bad_alloc();
1330   }
1331   if (void* p = std::malloc(n ? n : 1)) return p;
1332   throw std::bad_alloc();
1333 }
1334 void operator delete(void* p) noexcept { std::free(p); }
1335 void operator delete(void* p, std::size_t) noexcept { std::free(p); }
1336 #if defined(__GNUC__) && !defined(__clang__)
1337 #pragma GCC diagnostic pop
1338 #endif
1339
1340 namespace {
1341
1342 // Runs `write` with the next large allocation made to fail; returns true if it
1343 // threw std::bad_alloc.
1344 template <typename F>
1345 bool writeThrowsBadAlloc(F&& write) {
1346   bool threw = false;
1347   failinj::armed = true;
1348   try {
1349     write();
1350   }
1351   catch (const std::bad_alloc&) {
1352     threw = true;
1353   }
1354   failinj::armed = false;
1355   return threw;
1356 }
1357
1358 const std::string bigString(5000, 'x');   // built before any failure is armed
1359
1360 }  // namespace
```

- `failinj::armed` is a flag; while it is set, the first allocation of at least `failinj::threshold` (1 000)

bytes throws `std::bad_alloc` and clears the flag, so exactly one allocation fails. Deque nodes (512 bytes), the strings used as values and everything GoogleTest allocates are smaller than the threshold; only the copy of `bigString` (5 001 bytes with the terminator) is larger.

- `writeThrowsBadAlloc(f)` arms the flag, runs one write and reports whether `std::bad_alloc` was thrown; the flag is cleared again on every exit path.
- `bigString` is a namespace-scope constant, constructed before any test arms the flag.
- `InjectionWorks` asserts that the hook fires at all, so that the other tests cannot pass vacuously.
- The replacement `operator delete` calls `std::free`. At `-O2` and `-O3` GCC inlines the replaced `operator new` into callers such as `std::make_unique` and then reports `-Wmismatched-new-delete`, which is a false positive for a matched replacement pair. The file silences it with `#pragma GCC diagnostic ignored` around these three definitions only, guarded so that other compilers do not see it.

10.3 The failure tests

Test	Scenario and assertions
`MemvarFailedWrite.InjectionWorks`	the hook throws, and disarms itself
`.PlainMemvarIsUnchanged`	failed write on `memvar<string>`: size, both values, `m(2)` still throws; a later write works
`.FullHistoryDoesNotEvictTheOldestValue`	capacity 2, full: the failed write keeps the oldest value
`.CompoundAssignmentIsUnchanged`	a failing `m += bigString` changes nothing
`.DoesNotAgeOutstandingSafeRefs` (modes 0, 1)	a failed write does not count; the handle is still valid after the next good write
`MemvarTimedFailedWrite.HistoriesStayInStep`	after the failed write: size 1, same value, same tag, `getTimeTag(1)` and `getHistoryTimedValue(1)` throw; after a good write each value keeps its own tag
`.FullHistoryKeepsValuesAndTagsTogether`	full history of 3: nothing changes after the failure; a later write evicts value and tag together
`.AfterClearHistory`	after `clearHistory()` the tag is exactly zero and survives a failed write
`MemvarTimedEntries.ValueAndTagAreAlwaysPaired`	9 writes at 200 μs spacing into capacity 4: for each index, `getHistoryTimedValue(i)` agrees with `valueAt(i)` and `getTimeTag(i)`; newer tags are larger
`.TimeTagIndexIsCheckedLikeValueIndex`	`getTimeTag(-1)`, `getTimeTag(1)`, `getHistoryTimedValue(-1)` throw `out_of_range`; `getTimeTag(0)` does not

Table 11: The failure-injection and entry-pairing tests.

A representative test is shown in Listing 48.

Listing 48: HistoriesStayInStep: a failed write leaves value history and tags in step.

```
1426 TEST(MemvarTimedFailedWrite, HistoriesStayInStep) {
1427   mv::memvarTimed<std::string> m("first", 10);
1428   const auto tag0 = m.getTimeTag(0);
1429
1430   ASSERT_EQ(m.getHistorySize(), 1);
1431   EXPECT_TRUE(writeThrowsBadAlloc([&] { m = bigString; }));
1432
1433   // value history and time history both unchanged
```

```
1434    EXPECT_EQ(m.getHistorySize(), 1);
1435    EXPECT_EQ(m(), "first");
1436    EXPECT_EQ(m.getTimeTag(0), tag0);
1437    EXPECT_THROW((void)m.getTimeTag(1), std::out_of_range);    // used to succeed
1438    EXPECT_THROW((void)m.getHistoryTimedValue(1), std::out_of_range);
1439
1440    // a later good write pairs each value with its own tag
1441    m = "second";
1442    EXPECT_EQ(m.getHistorySize(), 2);
1443    EXPECT_EQ(m(0), "second");
1444    EXPECT_EQ(m(1), "first");
1445    EXPECT_EQ(m.getTimeTag(1), tag0);           // tag of "first", not of the
        failed write
1446    EXPECT_GE(m.getTimeTag(0), m.getTimeTag(1));
1447    EXPECT_THROW((void)m.getTimeTag(2), std::out_of_range);
1448
1449    const auto [v1, t1, e1] = m.getHistoryTimedValue(1);
1450    EXPECT_EQ(v1, "first");
1451    EXPECT_EQ(t1, tag0);
1452    EXPECT_FALSE(e1);
1453 }
```

10.4 Gaps and fragilities of the suite

These points follow from the test code; none is a defect of `memvar.h`.

- **Not covered:** output text (no test captures a stream, so the UTF-8 helpers, the `[ ... ]` and `[tag:value]` formats are untested), `getMemVarHistory()`, the wide and Unicode string types, mixed-type operands, a non-default `Clock` and any `Time` other than microseconds (one test), const variables, very large capacities, and any behaviour of the constraint system (rejected types are compile errors and cannot be tested by a compiled test).
- **No thread-safety tests**, consistent with the stated assumption that the header is not thread-safe.
- **Timing dependence.** `ValueAndTagAreAlwaysPaired` compares tags with a strict inequality after a 200 μs sleep and uses microsecond resolution; it relies on `steady_clock` advancing by at least a microsecond over a sleep, which holds on the test machine.
- **Cost.** `test_7` performs 10^9 writes and takes most of the suite's 3.7–4.3 s; `test_6` needs gigabytes of memory.
- **Program-wide replacement of `operator new`.** The injection hook applies to the whole test binary; the 1 000-byte threshold is an unexplained constant that would need revisiting if another test allocated a large object while the hook is armed (it is armed only around one statement, so none does).
- **Compiler warnings in the test file itself.** With `-Wall -Wextra`, `crefTest` produces `-Wunused-value` for the expression `*xcref + 1` inside `EXPECT_THROW` (mode 1, line 594) or `EXPECT_DEATH` (mode 0, line 599), and `-Wunused-but-set-variable` for `xcrefCopy` (modes 0 and 1, line 580). The default mode builds without warnings.

11 Limitations, Constraints, and Open Issues

This section separates *constraints* (deliberate restrictions that a user must know), from *open issues* (behaviours that are, in the view of this analysis, defects or traps). The evidence column of Table 12 says whether an item was **reproduced** by a program in this report, **measured**, or derived **by inspection** of the code only. The suggested directions are proposals; none has been implemented.

11.1 Constraints of the design

- **Value types** are limited to non-`bool` integrals, floating point, the five standard string types and `bip::bigint` (Section 2). The only extension point is specialising `mv::is_bigint`, and a new type must also provide the operators, the default constructor and `operator«` the header uses.
- **Capacity** is fixed per object at construction, at least 2, and stored as a signed 64-bit number.
- **Write guarantee:** every write (including compound assignment and `clearHistory`) has the strong exception guarantee, provided the copy/move constructor of `T` either succeeds or throws without side effects.
- **Type relationship:** `memvarTimed<T,...>` is not a `memvar<T>`; code that must accept both is written as a template constrained by `mv::IsMemvar` or over the operators.
- **No copying or moving** of a variable; value-only copy assignment.
- **No run-time polymorphism:** the destructors are not virtual and the classes are not `final`; a class derived from a variable must not be deleted through a pointer to it.
- **Not thread-safe**: neither the deque, the counters nor the time tags are synchronised. This is a stated assumption, not a defect.
- **History semantics:** index 0 is the newest value; history length is `min(writes, capacity)`; time tags are relative to a per-object epoch, which is the tag of the first value (reset by `clearHistory`).
- **C++20 required**; only g++ 13.3 with libstdc++ was exercised.
- **Include order:** `bigint.h` before `memvar.h`.
- **One value of `MEMVAR_SAFEREF_CHECK` per program.**

11.2 Summary of open issues

Table 12: Open issues and their evidence.

ID	Issue	Severity	Evidence	Where
I1	Edge holes: a const left operand of a compound assignment passes the constraint but fails in the body; capacities near `INT64_MAX` overflow in the checked modes	low	reproduced	§9
I2	Default mode gives no protection; `cref()` has a different type per mode	high	reproduced	§3.3
I3	Class layout depends on a macro (ODR)	medium	measured sizes	§7
I4	Arithmetic: overflow/division UB, silent truncation, mixed signedness (warning reported inside the header)	medium	reproduced / inspection	§9
I5	`deque` storage: 3 allocations per object, block churn on writes (worse for timed entries)	medium	measured	§7
I6	Implicit `operator T()` gives an inconsistent operator set and hidden copies	medium	reproduced	§6
I7	`operator«` is global: lookup hides it, output format changes silently	medium	reproduced	§9

ID	Issue	Severity	Evidence	Where
I8	Concept accepts types the classes cannot hold	low	reproduced	§2
I9	Inconsistent accessor names/return types; vestigial error flag; free `getHistoryTimedValue` drops the tag	low	reproduced	§4
I10	`clearHistory` resets to `T{}`; fixed capacity; no snapshot	low	inspection	§4
I11	Not copyable/movable: unsuitable for `vector`, no `swap`	low	inspection	§9
I12	Hard-wired `bigint` coupling and duplicate trait	low	inspection	§2
I13	Time-tag semantics (epoch, tag taken before the value is built, clock choice, truncation, unit-less print)	low	inspection	§5
I14	`SafeRef` limits (owner lifetime, deduction contexts, cost of throwing mode)	low	inspection	§3.3
I15	Global-namespace Unicode stream overloads	low	inspection	§6
I16	Verification limited to one toolchain; one unit test not runnable here	info	this report	§7
I17	`memvarTimed` is not a `memvar<T>`; implementation types leak (`TimedEntry`, `ValueInterface`); the hook contract of the mixin is unchecked	medium	reproduced	§9.6

11.3 Details

I1 – Holes at the edges of the constraints and of the counter arithmetic

- The compound-assignment templates require `IsMemvar<L>`, which is true for a const variable, so the overload is selected and the error appears inside `memvarAccess::set` instead of at the call (reproduced). A concept that required `L` to be non-const would give a clear diagnostic.
- `SafeRef` computes `writeCount + capacity` and `History::reset` adds the capacity to the counter; with a capacity near the maximum of `int64_t` both overflow in the checked modes (reproduced with UBSan). A bound on the capacity or a saturating addition would remove the case.

I2 – Default mode is unsafe, and `cref()` changes type with the mode

What happens. In the default mode `cref()` is a plain `const T&` into a deque slot that is destroyed after `capacity` further writes; using it afterwards is undefined behaviour with no diagnostic. In the checked modes the same call returns a `SafeRef`. Code that is written against one mode can compile and behave differently under another: `auto c = m.cref();` is a *copy* (safe) in the default mode but a *handle that can expire* in modes 0/1, and `decltype(m.cref())` differs.
Evidence. Listing 30 with its three outputs; Listing 36 prints `0` for "is a handle type" in the default mode.
Direction. Document that, in the default mode, any reference obtained from `cref()` is valid only until `capacity` further writes and that `auto` copies; or return a handle type in every mode whose check compiles to nothing in mode 2.

I3 – The macro changes the class layout (ODR)

`writeCount_` and the class `SafeRef` exist only in modes 0/1, so `sizeof(memvar<int>)` is 88 (mode 2) versus 96 (modes 0, 1) bytes, and `memvarTimed<int>` 96 versus 104. A program that links translation units compiled with different values has two different definitions of the same class template: undefined behaviour (ill-formed, no diagnostic required), typically observed as memory corruption. **Direction:** place each mode in a distinct inline namespace so that mixing produces a link error instead of silent corruption.

I4 – Arithmetic semantics are those of the built-in operators

- Signed overflow in `+ - *`, `++`, `-` and division or remainder by zero for integral `T` are undefined behaviour *(by inspection)*; `bigint` instead throws on division by zero.
- Compound assignment narrows silently: `memvar<int> += 1.9` records 2, `memvar<int8_t>(120) += 20` records −116 (reproduced).
- The binary operators return the *promoted* type (`int8_t + int8_t` is `int`), and permit mixed-type operands, so `memvar<int>(-1) < memvar<unsigned>(1)` is `false`; the compiler's `-Wsign-compare` fires, but inside `memvar.h` (reproduced), which makes it easy to suppress by mistake or to attribute to the library.

Direction: constrain compound assignment to results convertible to `T` without narrowing, and consider requiring both operands to have the same value type.

I5 – Storage cost and allocation behaviour of `std::deque`

With libstdc++ (512-byte nodes) a freshly built `memvar<int>(0,10)` makes 3 heap allocations and about 1 KiB, whatever the capacity, and so does a `memvarTimed<int>`. In steady state, *every* 128th write of an `int` (64th of a `long long`, 32nd of a timed `int`, whose entry is 16 bytes) frees a node and allocates another: 10^6 writes cost 7 812 (resp. 15 625 and 31 250) allocations even though the history never grows. The measured write cost (2.4 ns) is equal to a hand-written deque push/pop, so the overhead is that of the container choice, not of the wrapper. `getMemVarHistory()` additionally fixes `std::deque` as part of the public interface. **Direction:** a fixed-size ring buffer (one allocation, no churn, `capacity` known at construction) would remove both costs; the single `History` class is the only place that would change, apart from the return type of `getMemVarHistory()`.

I6 – The implicit `operator T()` makes the operator set inconsistent

Because of the non-explicit conversion, operators that the header does not define still compile for built-in `T`: `m % 3`, `-m`, bitwise operators, shifts (they act on a temporary copy, record nothing, and have no compound forms; `%` and unary minus were verified, the others follow by inspection). For `bigint`, whose `%` and unary minus are *member* operators, the same expressions do not compile, because member lookup is not helped by conversions (reproduced with the concepts `HasMod` and `HasNeg` in Listing 36). The conversion also copies: `std::string s = m;` copies the whole string, and all of `operator()`, `operator[]`, `valueAt` and `getHistoryValue` return by value, which matters for large strings and bigints; the only copy-free read is `cref()` or `getMemVarHistory()`. **Direction:** declare the missing operators explicitly or make the conversion `explicit`; offer reference-returning accessors.

I7 – The stream operators are in the global namespace

Argument-dependent lookup cannot find them, so in any scope that sees another `operator«` first (for instance a namespace that overloads it for its own types) they are hidden. For `memvar<int>` the code still compiles through the implicit conversion and prints the bare value; for `memvarTimed<int>` the output silently changes from `[0:3]` to `3`; for `memvar<std::wstring>` compilation fails (List-

ing 44). **Direction:** define them as hidden friends or in namespace `mv`.

I8 – `MemVarType` accepts `const T` and references

The concept strips cv-qualifiers and references before testing, but the history stores `std::deque<Entry>` with the original `T`. `memvar<const int>` and `memvar<int&>` pass the constraint and then fail with errors inside `<bits/stl_deque.h>` or in the mixin (reproduced). The error text does not name the real cause. **Direction:** also require `std::same_as<T, std::remove_cvref_t<T>>`.

I9 – Inconsistent accessor names, results and diagnostics

- `getHistoryValue(i)` (member) returns `tuple<T,bool>` while `mv::getHistoryValue(m, i)` returns `T`.
- The `bool` of the tuple is always `false`: out-of-range indices throw, so the "error flag" carries no information.
- `memvarTimed::getHistoryTimedValue(i)` returns the tuple with the time tag; `mv::getHistoryTimedValue(mvt, i)` returns only the value (it calls `mvt(i)`).
- `valueAt`, `operator()(i)` and `operator[]` are three spellings of one operation.

I10 – Clearing and capacity

`clearHistory()` resets the *current value* to `T{}`; there is no way to drop the history but keep the latest value, other than writing it again, and every outstanding `SafeRef` expires at the same time. The capacity cannot change after construction and cannot be less than 2; a capacity of 1 would be a meaningful degenerate case. There is no way to take a snapshot of the history as a value other than copying `getMemVarHistory()`.

I11 – Not copyable, not movable

Deleted copy/move construction and move assignment prevent `std::vector<memvar<T>>`, returning a named local, swapping, and aggregating memvars into a copyable struct. The value-only copy assignment may surprise: `a = b` between two variables of different capacity records one value and keeps `a`'s capacity.

I12 – Coupling to `bip::bigint`

The header hard-codes the name `bip::bigint` (forward declaration and trait specialisation), duplicates the trait that already exists as `bip::is_bigint`, and is correct only if `bigint.h` was included first. A different big-integer type can be admitted only by editing the header or specialising `mv::is_bigint`, and then must supply operator overloads that match the generic ones (for example `T + int`, used by ++).

I13 – Time-tag semantics

The epoch is the tag of the first entry, which therefore reads as exactly zero. Tags are offsets from a per-object epoch, so they are not comparable between objects or across a `clearHistory()`. The clock is read before the value is constructed (it is an argument of `History::push`), so for a large `T` the tag marks the start of the write rather than its commit. `Time` truncates (milliseconds lose sub-millisecond differences); a non-monotonic `Clock` can yield decreasing tags; printed tags carry no unit.

I14 – Limits of `SafeRef`

It protects only against entry eviction, not against destruction of the owner (raw pointer to the counter); implicit conversion fails in contexts that need template deduction or a member operator, so `*h` is needed there; `ABORT` mode terminates the whole process on a stale handle (appropriate for a test or debug configuration, hard to recover from in production); `THROWS` mode reintroduces the unwinding overhead the other mode avoids.

I15 – Overloads injected into the global namespace

The wide-string `operator«` templates are in the global namespace and apply to *every* `std::ostream`, not just to the header's own use. A program or another library that defines the same overloads gets ambiguity; invalid code units are silently replaced by U+FFFD, and there are no overloads for wide output streams.

I16 – Verification limits of this report

Only g++ 13.3/libstdc++ was used for the measurements; sizes and allocation counts are specific to it. Other GCC versions, `clang++`, MSVC and other standard libraries were not available. The unit test `memVarTest.test_6` (one-billion-entry history) was not run. Undefined-behaviour statements in I4 were derived from the code, not triggered. AddressSanitizer and UBSan were run on the unit tests and examples with no report; ThreadSanitizer was not used, since thread safety is out of scope.

I17 – Consequences of the CRTP structure

- `memvarTimed<int>` does not convert to `const memvar<int>&` (checked with `static_assert`, Listings 36 and 42); a function written for the plain class cannot be handed a timed variable, and `memvarTimed<int, ms>` and `memvarTimed<int, ns>` do not share a base either.
- `getMemVarHistory()` and the public alias `Entry` expose `mv::detail::TimedEntry`, and the base class of both variables is a `detail::ValueInterface` instantiation, so names from a `detail` namespace are part of the user-visible interface and of compiler diagnostics.
- The mixin's contract with its derived class (a private `setValue(U&&)` reachable through `friend Base`, an `Entry` whose value can be read by `valueIn`) is written nowhere in code; a wrong `Derived` produces an error inside the mixin rather than a constraint failure.
- The classes are neither `final` nor have virtual destructors, and the destructor of the mixin is protected: deleting a class derived from a variable through a base pointer would be undefined behaviour.

Direction: add a `concept` for the derived-class contract; either give both flavours a common non-template base that carries the read-only interface, or document that generic code takes a type constrained by `mv::IsMemvar`; move `TimedEntry` out of `detail` or hide it behind an accessor that returns value/tag views; mark the two classes `final`.

12 Conclusion

`memvar.h` implements a small, coherent idea: a variable that keeps its own bounded past, with ordinary-looking syntax. The implementation is compact: one history component holds every invariant of a write, one CRTP mixin gives both variable classes the same read, assignment, increment and printing interface, `memvarTimed` reuses all of it by choosing a different entry type and adding a time tag and an epoch, a single constrained template serves each operator, and a build-time switch turns a cheap generation-counter check against dangling history references on or off. Measurements show that the wrapper adds no measurable cost over the deque it wraps, that the checked modes cost nothing noticeable in this benchmark, and that nothing on any path is virtual. A write has the strong exception guarantee, and unit tests that inject an allocation failure pin that behaviour down for plain and timed variables, full histories

and `clearHistory()`.

The remaining weaknesses are at the edges: the default build's lack of protection together with a mode-dependent `cref()` type, a class layout that depends on a macro, an operator surface that mixes explicit templates with implicit conversions, stream operators that sit in the global namespace, a storage choice (`std::deque`) whose allocation behaviour is worse than the problem requires, and a timed class that is not a `memvar<T>`. If changes were prioritised, the order suggested by severity and cost would be (1) tighten `MemVarType` (I8) and put the safety modes in distinct inline namespaces (I3), (2) decide how generic code should refer to both flavours and constrain the mixin's contract (I17), (3) move the stream operators into namespace `mv` (I7), (4) replace the deque by a ring buffer (I5), and (5) decide whether the implicit conversion should remain (I6).